A Major Project Report

On

TRANSFER LEARNING BASED ENSEMBLE CLASSIFIER FOR MULTI-CLASS TEXTURE CLASSIFICATION IN REMOTE SENSING

Submitted

to

Kommuri Pratap Reddy Institute of Technology (Autonomous) Affiliated to JNTUH in partial fulfillment of the Requirement for the Award of the Degree of

Bachelor of Technology

in

Computer Science and Engineering (AI&ML)

by

SYEDA SANOBAR ALI (22RA1A6673)

PATIMEENI SHIVA MUKESH (22RA1A6676)

ALVAL ABHILASH GOUD (23RA1A6601)

Under the guidance of

Mrs. G. Sujatha,

M.Tech.(Ph.D)

Assistant Professor CSE(AI&ML)

KPRIT

Imagine. Innovate. Inspire.

Autonomous

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (AI&ML)

KOMMURI PRATAP REDDY INSTITUTE OF TECHNOLOGY

(An UGC Autonomous Institution, Affiliated to JNTUH, Hyderabad)

Ghanpur (V), Ghatkesar (M), Medchal (Dist.,)-500088

2022-2026

KOMMURI PRATAP REDDY INSTITUTE OF TECHNOLOGY

(An UGC Autonomous Institution, Affiliated to JNTUH, Hyderabad)

Ghanpur (V), Ghatkesar (M), Medchal (D)-500088

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING(AI&ML)



CERTIFICATE

This is to certify that the dissertation entitled **"TRANSFER LEARNING BASED ENSEMBLE CLASSIFIER FOR MULTI-CLASS TEXTURE CLASSIFICATION IN REMOTE SENSING"** being submitted by **S.SANOBAR ALI (22RA1A6673), P.SHIVA MUKESH (22RA1A6676), A.ABHILASH GOUD (23RA5A6601)** in partial fulfillment for the award of Bachelor of Technology in Computer Science and Engineering (AI&ML) to the Kommuri Pratap Reddy Institute of Technology is a record of confined work carried out by them under my guidance and supervision.

Internal Guide's signature
Mrs. G. SUJATHA
M.Tech.,Ph.D.
Assistant Professor
Dept. of CSE (AI&ML)

Signature of the Head of the Dept.
Dr.PULIME SATYANARAYNA
M.Tech.,Ph.D.
Professor
Dept. of CSE (AI&ML)

External Examiner

Place: Ghanpur
Date:

KOMMURI PRATAP REDDY INSTITUTE OF TECHNOLOGY

(An UGC Autonomous Institution, Affiliated to JNTUH, Hyderabad)

Ghanpur (V), Ghatkesar (M), Medchal (D)-500088

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING(AI&ML)



DECLARATION

We, **S.SANOBAR (22RA1A6673), S.MUKESH (22RA1A6676), A.ABHILASH (23RA5A6601)** hereby declare that the project report titled

"TRANSFER LEARNING BASED ENSEMBLE CLASSIFIER FOR MULTI-CLASS TEXTURE CLASSIFICATION IN REMOTE SENSING"

under the guidance of **Mrs. G SUJATHA**, Assistant Professor of Computer Science and Engineering(AI&ML), Kommuri Pratap Reddy Institute of Technology, Ghanpur, is submitted in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering (AI&ML).

This is a record of bonafide work carried out by us and the results embodied in this project have not been reproduced or copied from any source. The results embodied in this thesis have not been submitted to any other University and Institute for the award of any Degree or Diploma.

SUBMITTED BY

SANOBAR ALI	**(22RA1A6673)**
SHIVA MUKESH	**(22RA1A6676)**
ABHILASH GOUD	**(23RA5A6601)**

ACKNOWLEDGEMENT

The satisfaction and euphoria that accompany the successful completion of any task would be incomplete without acknowledging the individuals who made it possible.

We are extremely indebted to our **INTERNAL GUIDE**, **Mrs. G SUJATHA M.Tech.,Ph.D** Assistant Professor, for her valuable help. We would like to express our profound gratitude to her for providing us the opportunity to work under her supervision and guidance. Her guidance helped us in the successful completion of our project.

We would like to express our heartfelt gratitude to our **HoD, Dr. PULIME SATYANARAYANA, M.Tech.,Ph.D** Professor, for providing the guidance and necessary infrastructure to complete this project.

We would also like to express our deep-felt gratitude to our **DIRECTOR**, **Prof. B. SUDHEER PREM KUMAR, M.Tech.,Ph.D** of Kommuri Pratap Reddy Institute of Technology, for his kind cooperation and valuable suggestions.

We would also like to express our deep-felt gratitude to **Dr. SREENATH KASHYAP, M.Tech.,Ph.D., VICE PRINCIPAL** of Kommuri Pratap Reddy Institute of Technology, for his kind cooperation and valuable suggestions.

This Acknowledgement will be incomplete without mentioning our sincere gratefulness to our honorable **CHAIRMAN Shri. KOMMURI PRATAP REDDY,** and **VICE-CHAIRMAN Shri KOMMURI PRASHANTH REDDY** of Kommuri Pratap Reddy Institute of Technology for providing the necessary infrastructure to complete this project.

We also thank the staff of the CSE(AI&ML) department who helped us morally for the successful completion of this project.

SUBMITTED BY

SANOBAR ALI	**(22RA1A6673)**
SHIVA MUKESH	**(22RA1A6676)**
ABHILASH GOUD	**(23RA5A6601)**

Kommuri Pratap Reddy Institute of Technology
Department of Computer Science and Engineering
(Artificial Intelligence & Machine Learning)

Vision of the Institute:

To emerge as a premier institute for high quality professional graduates who contribute to economic and social developments of the Nation.

Mission of the Institute:

Mission	Statement
IM1	To have holistic approach in curriculum and pedagogy through industry interface to meet the needs of Global Competency.
IM2	To develop students with knowledge, attitude, employability skills, entrepreneurship, research potential and professionally Ethical citizens.
IM3	To contribute to advancement of Engineering & Technology that would help to satisfy the societal needs.
IM4	To preserve, promote cultural heritage, humanistic values and Spiritual values thus helping in peace and harmony in the society.

Vision of the Department:

To be a premier center of excellence with research in Artificial Intelligence and Machine Learning research through unique interdisciplinary partnerships and positive contribution to the students, community, organizations and society as a whole.

Mission of the Department:

Mission	Statement
DM1	To impart knowledge in cutting edge Artificial Intelligence technologies in par with industrial standards.
DM2	To collaborate with industry to uplift innovative research and development in Artificial Intelligence and Machine Learning and its allied fields to serve the needs of society.
DM3	To produce successful Computer Science and Engineering graduates with a specialization in AIML with personal and professional responsibilities and commitment to lifelong learning.

PROGRAM OUTCOMES

PO1	**Engineering Knowledge:** Apply the knowledge of mathematics, science, Engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
PO2	**Problem Analysis:** Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
PO3	**Design/development of Solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
PO4	**Conduct investigations of complex problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
PO5	**Modern tool usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
PO6	**The engineer and society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
PO7	**Environment and sustainability:** Understand the impact of the professional engineering solutions in societal and environmental knowledge of, and need for sustainable development.

PO8	**Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
PO9	**Individual and team network:** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
PO10	**Communication**: Communicate effectively on complex engineering activities with the engineering community and with society at large, being able to comprehend and write effective reports and design documentation, make Effective presentations, and give and receive clear instructions.
PO11	**Project management and finance**: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work,as a member and leader in a team, to manage projects and in multidisciplinary environment**.**
PO12	**Life-Long learning:** Recognize the need for, and have the preparation and able to engage in independent and life-long learning in the broadest context of technological change.

PROGRAM SPECIFIC OUTCOMES

PSO1	Demonstrate the knowledge of human cognition, Artificial Intelligence, Machine Learning and data engineering for designing intelligent systems.
PSO2	Apply computational knowledge and project development skills to provide innovative solutions.
PSO3	Use tools and techniques to solve problems in AI & ML.

ABSTRACT

Automated texture classification has become an important area of research in computer vision and industrial inspection systems, where accurate surface analysis is required to maintain product quality and detect defects during manufacturing. In earlier industrial practices, surface inspection was performed manually by human experts who visually examined materials to identify defects such as holes, cracks, and foreign objects. Although manual inspection was widely used, it was often slow, inconsistent, and prone to human error. Later, traditional automated systems based on classical image processing techniques were introduced, but these methods relied heavily on handcrafted features and simple classification approaches, which limited their ability to capture complex texture patterns. As industrial datasets became larger and more diverse, these limitations created a need for intelligent systems capable of extracting meaningful visual features and performing reliable multi-class classification. To address these challenges, this study presents a transfer learning–based texture classification framework that combines deep feature extraction with machine learning techniques. In the proposed approach, a pretrained Visual Geometry Group 19 (VGG19) is used to extract high-level visual features from industrial texture images. The extracted feature vectors are then used to train multiple machine learning classifiers including Linear Discriminant Analysis Classifier (LDAC), Quadratic Discriminant Analysis Classifier (QDAC), Support Vector Classifier (SVC), and Extra Trees Classifier (ETC). Among these models, the proposed VGG19 with ETC classifier achieved the highest classification accuracy of 97.31%, demonstrating its strong capability in distinguishing between different texture categories such as good, hole, and objects. The results highlight the effectiveness of combining deep feature extraction with ensemble machine learning techniques for reliable and efficient industrial texture classification.

KEYWORDS

Keyword	Definition
Remote Sensing	AI-driven analysis of sensor or satellite data to detect Earth patterns.
Texture Classification	Categorizing images based on patterns and visual textures.
Transfer learning	Reusing a pre-trained model for a new, related task.
Deep Neural Network	AI models with multiple layers learning complex data patterns.
TILDA-400 Dataset	Labeled image dataset for texture classification evaluation.
Feature Extraction	Selecting key patterns from raw data for AI models.
Data Augmentation	Increasing dataset size by creating modified data versions.
Environmental Monitoring	Using technology to observe air, water, and land conditions.
Computational Efficiency	Performing tasks using minimal time and resources.

TABLE OF CONTENTS

DESCRIPTION	PAGE NO
ABSTRACT	I
LIST OF FIGURES	IV
LIST OF SCREENSHOTS	V
1 INTRODUCTION	**1**
1.1 Overview	2
1.2 Problem Definition	3
1.3 Research Motivation	2
1.4 Significance	3
1.5 Research Objective	3
1.6 Advantages	3
1.7 Applications	4
2 LITERATURE SURVEY	**6**
3 EXISTING SYSTEM	**11**
3.1 Traditional System	11
3.2 Limitations	12
4 PROPOSED SYSTEM	**14**
4.1 System Architecture	14
4.2 Data Preprocessing	17
4.3 VGG19	19
4.4 Model Building and Training	21
4.4.1: Linear Discriminant Analysis Classifier (LDAC)	21
4.4.2. Quadratic Discriminant Analysis Classifier (QDAC)	23
4.4.4. Support Vector Classifier (SVC)	25
4.4.5 Extra Trees Classifier (ETC)	26
5 UML DIAGRAMS	**29**
5.1 Class Diagram	30
5.2 Use Case Diagram	31
5.3 Sequence Diagram	32

5.4 Component Diagram 33
5.5 Deployment Diagram 34
5.6 Activity Diagram 35
5.7 Data Flow Diagram (DFD) 36
6 SOFTWARE ENVIRONMENT 38
6.1 Software Environment 38
6.1.1 Python Packages 39
6.2. Hardware Environment 42
7 FUNCTIONAL AND NON-FUNCTIONAL REQUIREMENTS 45
7.1 Functional Requirements 45
7.2 Non-Functional Requirements 46
7.3 System Study 47
8 SOURCE CODE 50
9 RESULTS AND DISCUSSION 54
9.1 Implementation description 55
9.2 Dataset Description 56
9.3 Result Description 57
9.4 Comparative Analysis 104
10 CONCLUSION AND FUTURE SCOPE 72
REFERENCES 73

LIST OF FIGURES

PAGE NO

Fig.1.1: Remote sensing scene classification _____ 4
Fig. 3.1: Architecture of Patch-Level Texture Classification System _____ 13
Fig. 4.1: System architecture of patch-level texture classification using vision transformers on TILDA-400. _____ 15
Fig. 4.2: Internal operation flow of data preprocessing for patch-level texture classification _____ 19
Fig. 4.3: Internal workflow of VGG19. _____ 20
Fig. 4.4: Internal operation flow of LDAC for patch-level texture classification ___ 22
Fig. 4.5: Internal operation flow of QDAC for patch-level texture classification ___ 24
Fig. 4.6: Internal operation of SVC for patch-level texture classification _____ 25
Fig. 4.6: Internal operation flow of ETC for patch-level texture classification _____ 27
Fig. 5.1: Class diagram for patch-level texture classification_____ 29
Fig. 5.2: Use case diagram for patch-level texture classification _____ 30
Fig. 5.3: Sequence diagram for patch-level texture classification _____ 31
Fig. 5.4: Component diagram for patch-level texture classification_____ 32
Fig. 5.5: Deployment diagram for patch-level texture classification _____ 33
Fig. 5.6: Activity diagram for patch-level texture classification _____ 34
Fig.5.7: Data flow diagram (DFD) for patch-level texture classification_____ 35

LIST OF SCREENSHOTS

PAGE NO

Screenshot. 9.1: Sample image from the good class 83

Screenshot. 9.2: Sample image from the hole class 84

Screenshot. 9.3: Sample image from the objects class 85

Screenshot. 9.4: GUI of the Texture Classification System 86

Screenshot. 9.5: Administrator Authentication Interface (Login and Signup) 87

Screenshot. 9.6: Administrative Control Dashboard for System Operations 87

Screenshot. 9.7: Dataset Selection and Upload Interface 88

Screenshot. 9.8: VGG19 Feature Extraction Output Display 88

Screenshot. 9.9: Dataset Train–Test Splitting Output 89

Screenshot. 9.10: Performance Evaluation of Existing LDAC Model 90

Screenshot. 9.11: Performance Evaluation of Existing QDAC Model 91

Screenshot. 9.12: Performance Evaluation of Existing SVM Model 92

Screenshot. 9.13: Performance Evaluation of Existing ETC Model 93

Screenshot. 9.14: Test Image Prediction Interface (Laptop 2 Execution) 94

Screenshot. 9.15: Prediction Result – Classified as Hole 94

CHAPTER 1

INTRODUCTION

1.1 Overview

Remote sensing (RS) is the science of collecting information about objects without any direct physical contact, typically through a satellite, aircraft or unmanned aerial vehicle (UAV). Examples of applications for remote sensing include geological surveys, environment testing, oil exploration, traffic management, earthquake prediction, and water conservancy construction. Remote-sensing images have improved in both spatial and temporal resolutions with the evolution of satellite sensors, which provides opportunities in resolving fine details on the earth's surface as shown in Fig 1.1 [1].

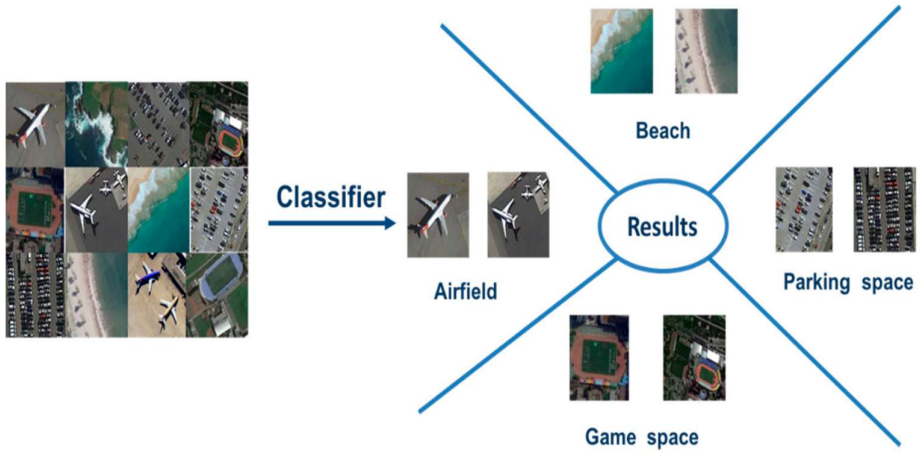


Fig.1.1: Remote sensing scene classification.

Satellites such as MODIS (1 km × 1 km) offering thermal data with high temporal resolution suffer from low spatial resolution. Landsat, on the other hand, offers small-scale variations of 100–200 m but with very low temporal resolution. The new generation of satellites can deliver very high spectral and spatial images; for example, IKONOS-2 generates images with 4-band multispectral resolution and spatial resolution from 2.5 to 4 m. Unmanned aerial vehicles (UAVs) present an improved solution of remote-sensing acquisition platforms, which witnessed a high level of growth in past years and are used widely for fire detection, surveillance mapping, and landslide monitoring, among other uses [2]. UAVs have several advantages over satellite and aerial images. First, they are easier to deploy to satisfy the requirements of

rapid monitoring, assessment, and mapping. They can work at lower altitudes compared to the piloted aircraft, which provides spatial resolution at the centimeters level. They can fly any time the weather permits, leading to improvements in temporal resolution. As the spatial resolution increases, images are likely to contain noisy and outlying descriptors. A recent study [3] proposed a convolutional neural network (CNN) to classify images captured by a camera mounted on a UAV. CNN model to classify a digital surface model beside UAV images [4] combined CNNs with object-based image analysis (OBIA) for land cover classification using Multiview data. The two-branch neural network to assign multiple class labels to UAV imagery [5].

1.2 Problem Definition

Industrial surface inspection involves analysing images to identify defects such as holes, cracks, and foreign objects that may occur during manufacturing. Traditional inspection methods rely on manual observation or simple image processing techniques, which often struggle to handle complex textures and variations in lighting conditions. These approaches may fail to detect subtle defects or may incorrectly classify normal surfaces as defective. Additionally, large production environments generate thousands of inspection images daily, making manual analysis inefficient and impractical. The complexity and high dimensionality of image data create challenges in accurately extracting meaningful patterns for classification. Therefore, there is a need for effective image analysis approaches capable of reliably identifying defects within large-scale industrial datasets.

1.3 Research Motivation

In modern manufacturing industries, maintaining product quality is a critical requirement to ensure customer satisfaction and operational efficiency. Large-scale companies such as automobile manufacturers, electronics producers, and semiconductor industries generate massive volumes of visual data through inspection cameras and monitoring systems during production. Analysing this image data manually is time-consuming and often leads to inconsistencies due to human error. Therefore, data analysis techniques have become essential for processing and interpreting large datasets efficiently. Advanced image analysis enables industries to identify defects, monitor surface conditions, and improve quality control processes. The growing reliance on automated inspection and data-driven decision-making motivates

the development of intelligent systems capable of analysing industrial texture images accurately and efficiently.

1.4 Significance

The significance of this research lies in its contribution to improving automated visual inspection systems used in industrial environments. Accurate analysis of surface textures helps industries detect defects early in the production process, reducing waste and preventing faulty products from reaching customers. Automated image analysis also enhances consistency and reliability compared to manual inspection methods. By utilizing data analysis techniques for image classification, industries can process large volumes of visual data efficiently and maintain high product quality standards. Furthermore, such systems support modern smart manufacturing environments where intelligent monitoring and quality control are essential for improving production efficiency and operational performance.

1.5 Research Objective

1. To extract meaningful visual features from industrial texture images using the VGG19 transfer learning model for effective representation of surface patterns.
2. To implement machine learning classifiers including LDAC, QDAC, SVC, and ETC for multi-class texture classification.
3. To evaluate the performance of the classifiers using metrics such as accuracy, precision, recall, F1-score, confusion matrix, and Receiver Operating Characteristic- Area Under the Curve (ROC-AUC) analysis to determine the effectiveness of the classification system.

1.6 Advantages

- Enables fine-grained analysis of remote sensing images by classifying textures at the patch level, improving accuracy in localized decision-making.
- Reduces dependency on manual annotation and visual inspection by automating the detection of subtle surface changes.
- Allows scalable processing of high-resolution satellite and drone images using pre-trained architectures and optimized models.

- Enhances real-time detection capabilities for critical tasks such as road damage assessment, crop health monitoring, and pollution tracking.
- Provides better generalization across different environmental conditions due to robust model training on diverse datasets like TILDA-400.
- Lowers computational complexity during inference through efficient feature extraction and ensemble classifiers.
- Increases interpretability and reliability by offering visual outputs like confusion matrices and ROC curves for every model.

1.7 Applications

- Precision agriculture for identifying crop diseases, weeds, and soil textures from drone or satellite imagery.
- Road and infrastructure monitoring by detecting cracks, potholes, and surface wear from patch-level image classification.
- Environmental monitoring for changes in vegetation, erosion, or water body textures over time.
- Oil and gas industry applications in detecting surface anomalies, spills, or pipeline integrity issues from aerial inspections.
- Urban development analysis by classifying land use textures, building materials, and illegal construction patterns.
- Disaster management through quick classification of affected regions based on changes in surface textures.

CHAPTER 2

LITERATURE SURVEY

Bazi, et al. [6] proposed a remote-sensing scene-classification method based on vision transformers. These types of networks, which are now recognized as state-of-the-art models in natural language processing, do not rely on convolution layers as in standard convolutional neural networks (CNNs). Instead, they use multihead attention mechanisms as the main building block to derive long-range contextual relations between pixels in images. In a first step, the images under analysis are divided into patches, then converted to sequence by flattening and embedding.

Carrilho, et al. [7] reviewed; they summarized the most important advancements in the last 5 years and focus mostly on machine learning-based approaches. They also outline the most promising avenues of research in the future.

Wang, et al. [8] developed and found that transformers need more parameters than CNNs. Additionally, further research is also needed regarding inference speed to improve transformers' performance. It was determined that the most common application scenes for transformers in our database are urban, farmland, and water bodies. They also found that transformers are employed in the natural sciences such as agriculture and environmental protection rather than humanities or economics. Finally, this work summarizes the analysis results of transformers in remote sensing obtained during the research process and provides a perspective on future directions of development.

Wang, et al. [9] introduced the fundamental concepts of transformers and highlight the first successful Vision Transformer (ViT). Building on the ViT, they reviewed subsequent improvements and optimizations introduced for image classification tasks. They then compare the strengths and limitations of these transformer-based models against classic CNNs through experiments. Finally, they explored key challenges and potential future directions for image classification transformers.

Aleissaee, et al. [10] reviewed the remote sensing community and has also witnessed an increased exploration of vision transformers for a diverse set of tasks. Although a number of surveys have focused on transformers in computer vision in general, to the

best of their knowledge they are the first to present a systematic review of recent advances based on transformers in remote sensing. Their survey covers more than 60 recent transformer-based methods for different remote sensing problems in sub-areas of remote sensing: very high-resolution (VHR), hyperspectral (HSI) and synthetic aperture radar (SAR) imagery. They concluded the survey by discussing different challenges and open issues of transformers in remote sensing.

Tombe, et al. [11] presented a comprehensive review of the developments of various computer vision methods in remote sensing. There is currently an increase of remote sensing datasets with diverse scene semantics; this renders computer vision methods challenging to characterize the scene's images for accurate scene classification effectively. This paper presented technology breakthroughs in deep learning and discusses their artificial intelligence open-source software implementation framework capabilities. Further, this paper discusses the open gaps/opportunities that need to be addressed by remote sensing communities.

Mao, et al. [12] presented this work comprehensively comparing the entire YOLO family, highlighting key innovations and their practical implications. They also discuss the challenges, including dataset limitations, domain generalization, and computational constraints, proposing future solutions such as synthetic data generation, federated learning, and edge AI deployment. By bridging the gap between academic advancements and industrial applications, this review is a practical guide for selecting and optimizing YOLO models for fabric inspection, paving the way for intelligent quality control systems.

Zhang, et al. [13] proposed a new remote sensing scene classification method, Remote Sensing Transformer (TRS), a powerful “pure CNNs → Convolution + Transformer → pure Transformers” structure. First, they integrate self-attention into ResNet in a novel way, using our proposed Multi-Head Self-Attention layer instead of 3 × 3 spatial revolutions in the bottleneck. Then they connect multiple pure Transformer encoders to further improve the representation of learning performance completely depending on attention. Finally, use a linear classifier for classification. They train our model on four public remote sensing scene datasets: UC-Merced, AID, NWPU-RESISC45, and OPTIMAL-31. The experimental

results show that TRS exceeds state-of-the-art methods and achieves higher accuracy.

Li, et al. [14] investigated unexplored ideas for remote sensing image captioning tasks, using a novel patch-level region-aware module with a multi-label framework. Due to an overhead perspective and a significantly larger scale in RSIs, a patch-level region-aware module is designed to filter the redundant information in the RSI scene, which benefits the Transformer-based decoder by attaining improved image perception. Technically, the trainable multi-label classifier capitalizes semantic features as supplementary to the region-aware features. Moreover, modeling the inner relations of inputs is essential for understanding the RSI.

Zhang, et al. [15] proposed a multiple hierarchical cross-scale Transformer model that efficiently combines the Transformer model with CNNs and is specifically designed for complex remote sensing scene classification. Firstly, a feature pyramid network with attention aggregation extracts the multi-scale base features. Then, these base features are fed into the proposed multi-scale channel Transformer (MSCT) module to derive the global features with channel-wise attention. Additionally, the base features are also fed into the proposed hierarchical cross-scale Transformer (HCST) module, which can obtain multi-level cross-scale representations.

Lu, et al. [16] proposed a novel aerial object detection framework called DFCformer. DFCformer is mainly composed of three parts: the backbone network DMViT, which introduces deformation patch embedding and multi-scale adaptive self-attention to capture sufficient features of the objects; FRGC guides feature interaction layer by layer to break the barriers between feature layers and improve the information discrimination and processing ability of multi-scale critical features; CAIM adopts an attention mechanism to fuse multi-scale features to perform hierarchical reasoning on the relationship between different levels and fully utilize the complementary information in multi-scale features. Extensive experiments have been conducted on the FAIR1M dataset, and DFCformer shows its advantages by achieving the highest scores with stronger scene adaptability.

Basit, et al. [17] proposed a new semantic segmentation model based on Transformer, which is called Segmentation for Mine SegMine and consists of a Vision Transformer-based encoder and a lightweight attention mask decoder. The experimental results show

that SegMine enhances the network's ability to obtain local spatial detail information and improves the problem of disappearing small-scale object features and insufficient information expression. It also better preserves the boundary details of open-pit mining areas. Using the metrics of mIoU, precision, recall, and dice, experimental areas were selected for comparative analysis, and the results show that the new method is significantly better than six other existing major Transformer variants.

Marjani, et al. [18] trained CNN with multiple feature extractors for pixel-wise classification and introduced a new loss function, namely, "gradient profile" (GP) loss, which is in fact the constituent of the more generic spatial profile loss proposed for image translation problems. To training, testing, and performance evaluation, they use a publicly available dataset with selected oil spill events verified by the European Maritime Safety Agency (EMSA). The results obtained show that the proposed CNN trained with a combination of GP, Jaccard, and focal loss functions can detect oil spills with an intersection over union (IoU) value of 63.95%. The IoU value for sea surfaces, lookalikes, ships, and land class is 96.00%, 60.87%, 74.61%, and 96.80%, respectively.

Smith, et al. [19] aimed to address these challenges by proposing CVTNet, a novel deep learning (DL) model that integrates CNNs and ViT architectures. CVTNet uses channel attention (CA) and spatial attention (SA) mechanisms to enhance feature extraction from Sentinel-1 (S1) and Sentinel-2 (S2) satellite data. The primary goal of this model is to achieve a balanced trade-off between Precision and Recall, which is essential for accurate wetland mapping. The class-specific analysis demonstrated CVTNet's proficiency across diverse classes, including pasture, shrubland, urban, bog, fen, and water. Comparative analysis showed that CVTNet outperforms contemporary algorithms such as Random Forest (RF), ViT, multi-layer perceptron mixer (MLP-mixer), and hybrid spectral net (HybridSN) classifiers.

Zheng, et al. [20] proposed the application of a modern vision transformer (ViT) architecture for the purposes of low-resolution image-based anomaly detection for defect localisation as a means of leather surface defect classification. Utilising the low-resolution defective and non-defective images found in the open source Leather Defect detection and Classification dataset and higher-resolution MVTec AD anomaly benchmarking dataset, three configurations of the vision transformer and three deep

learning (DL) knowledge transfer methods are compared in terms of performance metrics as well as in leather defect classification and anomaly localisation.

Tao, et al. [21] developed a novel lightweight dual-branch swin transformer (LDBST) method for remote sensing scene classification, and the discriminative ability of scene features is increased through combining a ViT branch and convolutional neural network (CNN) branch. First, based on the hierarchical swin transformer model, LDBST divides the input features of each stage into two parts, which are then separately fed into the two branches. For the ViT branch, a dual multilayer perceptron structure with a depthwise convolutional layer, termed Conv-MLP, is integrated into the branch to boost connections with neighboring windows.

CHAPTER 3

EXISTING SYSTEM

3.1 Traditional System

In earlier industrial inspection environments, surface texture analysis was primarily performed using manual inspection methods and basic image processing techniques. Skilled operators visually examined industrial materials to identify defects such as holes, scratches, cracks, or foreign objects present on the surface. In some cases, conventional computer vision systems were used to automate inspection tasks. These systems relied on handcrafted feature extraction methods such as edge detection, histogram-based texture analysis, and statistical texture descriptors. The extracted features were then used with simple classification algorithms to identify surface conditions. Although these approaches could detect obvious defects, they often struggled to handle complex texture patterns and variations present in real-world industrial images.

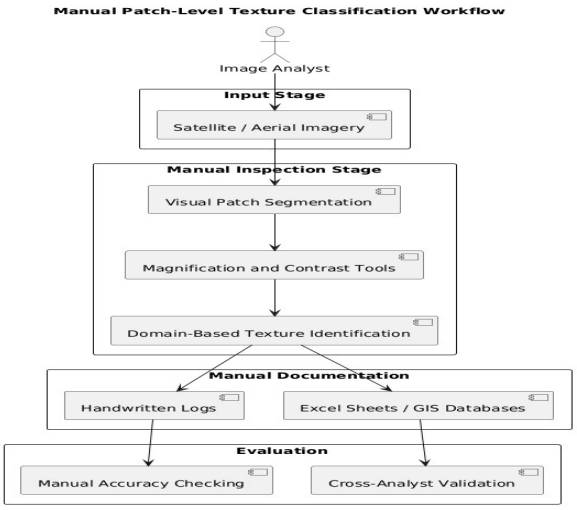


Fig. 3.1: Architecture of Traditional Manual Patch-Level Texture Classification System.

Traditional inspection systems typically depended on fixed feature extraction techniques that were manually designed by domain experts presented in fig 3.1. These handcrafted features were sensitive to lighting conditions, noise, and surface irregularities, which reduced their reliability when applied to large and diverse datasets.

Additionally, traditional machine learning classifiers often require carefully engineered feature representations to achieve acceptable performance. Because of these limitations, conventional systems could not accurately capture high-level visual patterns present in texture images. As a result, their classification performance was limited when compared to modern deep learning–based approaches.

3.2 Limitations

- Highly time-consuming and labour-intensive, requiring human supervision for every image patch.
- Subjective in nature; different analysts could produce different classifications for the same patch.
- Not scalable for large datasets or real-time satellite imagery streams.
- Difficult to maintain consistency in classification across multiple analysts or time periods.
- No provision for learning or adapting from past classification mistakes or feedback.
- Manual segmentation and patch extraction lacked precision and consistency.
- High chance of missing subtle texture variations due to visual fatigue or human oversight.
- Limited use of quantitative metrics; evaluations were often based on visual agreement.

CHAPTER 4

PROPOSED METHODOLOGY

4.1 System Architecture

The proposed methodology follows a structured framework for developing an intelligent image analysis system capable of performing multi-class texture classification in industrial environments shown in fig. 4.1. The methodology begins with dataset acquisition and image preprocessing, followed by deep feature extraction using transfer learning techniques. These extracted features are then utilized by multiple machine learning classifiers to analyse surface textures and identify different defect categories. The system integrates classical classification models with deep feature extraction to improve classification accuracy and reliability. A graphical interface allows users to interact with the system, perform dataset uploads, execute model training, and view classification results. This structured workflow ensures efficient data handling, model evaluation, and prediction generation for automated texture analysis.

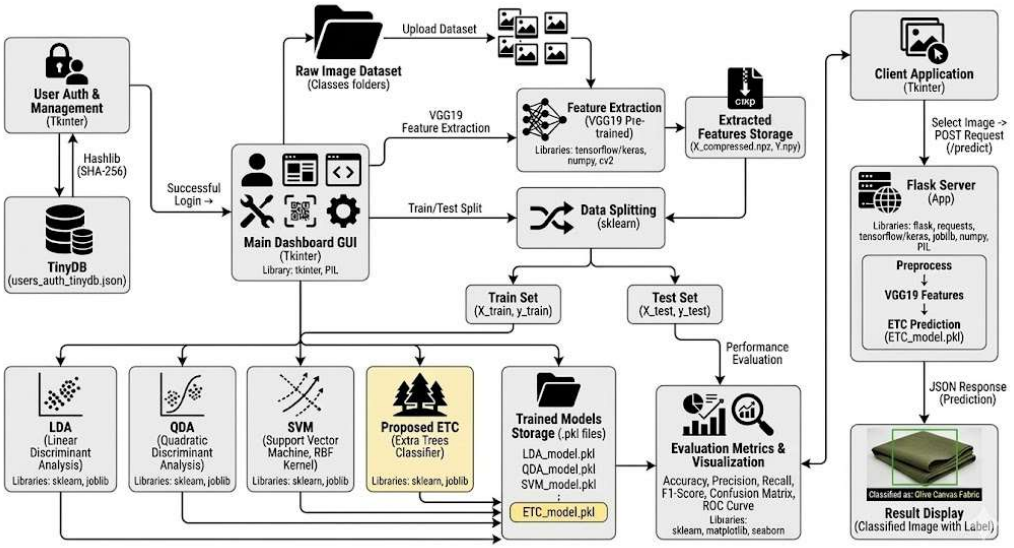


Fig. 4.1: System architecture of patch-level texture classification using vision transformers on TILDA-400.

User Interface (Tkinter GUI)

- The user interacts with the system through a custom graphical interface developed using Tkinter.
- The interface facilitates critical workflow stages, including dataset uploading, feature extraction, model training, and performance evaluation.
- Users can visualize real-time classification results, confusion matrices, and detailed performance metrics.
- All GUI-triggered actions are communicated to backend modules that orchestrate the data analysis pipeline.

Dataset Input (Image Dataset)

- The system utilizes a structured dataset of industrial texture images categorized into specific class folders.
- These categories represent surface conditions such as normal surfaces, holes, or foreign objects.
- Images are loaded into the system and prepared for the preprocessing pipeline to ensure consistency across the dataset.

Image Preprocessing

- To ensure uniform dimensions for the neural network, all uploaded images are resized to a standard resolution.
- Images are converted into multidimensional arrays and normalized to optimize the efficiency of the feature extraction process.
- This standardized data format is then forwarded to the deep learning feature extractor.

Feature Extraction using VGG19

- A pretrained **VGG19** convolutional neural network serves as the backbone for capturing high-level visual patterns.
- The final fully connected layers are truncated, allowing the convolutional base to act as a pure feature extractor.

- **Global Average Pooling** is applied to the output to generate compact numerical feature vectors that represent structural details and spatial patterns.

Train–Test Data Splitting

- The extracted feature vectors are partitioned into distinct training and testing subsets.
- The training set allows the classifiers to learn specific texture patterns, while the testing set provides an unbiased evaluation of the models' predictive power.
- This separation is crucial for validating the generalizability of the system on unseen industrial data.

Existing Baseline Models (LDAC, QDAC, SVC)

- The feature vectors are processed by several classical statistical and machine learning classifiers to establish a performance benchmark:
    - **LDAC:** Focuses on maximizing the linear separability between different surface classes.
    - **QDAC:** Models are more complex, quadratic decision boundaries for higher-dimensional data.
    - **SVC:** Utilizes kernel functions to identify the optimal hyperplane that separates texture categories.

Proposed Model (ETC)

- ETC is implemented as the core ensemble learning engine for the framework.
- It operates by constructing many randomized decision trees and aggregating their results to reach a final classification.
- By using randomized feature selection during tree construction, the model effectively reduces overfitting and improves the overall accuracy of texture classification.

Performance Evaluation and Result Analysis

- The outputs of both the baseline and proposed models are rigorously analyzed using a comprehensive suite of metrics.
- Evaluation includes Accuracy, Precision, Recall, and F1-score, alongside visual tools like Confusion Matrices and ROC Curves.
- This analysis identifies the most reliable classifier for specific industrial environments and surface types.

Prediction and Output Visualization

- In the final stage, the trained classifier is deployed to identify the class of new, unseen industrial images.
- New inputs pass through the same preprocessing and VGG19 feature extraction pipeline.
- The predicted class label is overlaid on the image and displayed within the Tkinter UI, allowing for rapid identification of surface conditions and defects.

4.2 Data Preprocessing

Data preprocessing plays a crucial role in preparing raw image data for further analysis and classification. In this study, preprocessing ensures that the input images are transformed into a consistent and suitable format before feature extraction and model training. Industrial texture images may vary in size, format, and quality, which can affect the performance of the classification models. Therefore, preprocessing operations are performed to standardize the images, remove inconsistencies, and convert them into numerical representations that can be processed by machine learning algorithms. These operations improve the reliability of the extracted features and enhance the overall performance of the classification process.

Image Loading: Fig 4.2, The first step involves loading texture images from the dataset directory. Each class of images is organized into separate folders, allowing the system to automatically assign class labels during loading. The images are read and prepared for further processing.

Image Resizing: All images are resized to a fixed dimension of 224 × 224 pixels. This standardization ensures that every image has the same spatial resolution required for the feature extraction model. Uniform image size also simplifies further computational processing.

Image Conversion to Numerical Arrays: After resizing, each image is converted into a numerical array representation. This conversion transforms pixel values into a matrix format that can be processed by deep learning frameworks. Numerical arrays enable efficient manipulation of image data during analysis.

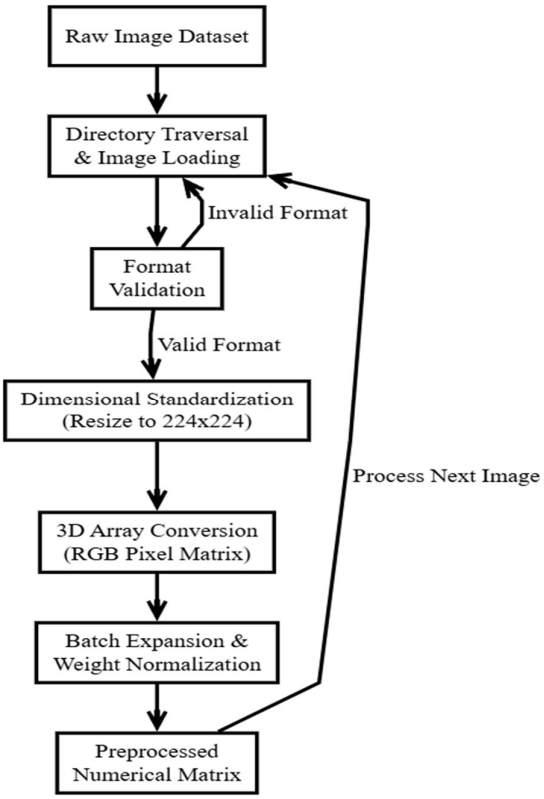


Fig. 4.2: Internal operation flow of data preprocessing for patch-level texture classification

Image Normalization and Preprocessing: The pixel values of the images are normalized using preprocessing functions. This step adjusts the image intensity values to a standardized range suitable for deep neural network input. Normalization improves model stability and enhances feature extraction quality.

Feature Vector Preparation: The processed images are converted into feature-ready inputs by expanding their dimensions and organizing them into structured arrays. These feature-ready inputs allow the feature extraction model to analyze the images effectively. The prepared image data is then forwarded to the feature extraction stage for further processing.

4.3 VGG19

The feature extraction stage plays a crucial role in transforming raw image data into meaningful representations that can be used for classification. In this study, a pretrained deep convolutional neural network is used to extract high-level visual features from industrial texture images. The network processes the input images through multiple convolutional and pooling layers to capture important spatial patterns such as edges, shapes, and texture structures. Instead of using the network for direct classification, the final classification layers are removed so that the deep network functions as a powerful feature extractor. The resulting feature vectors represent the visual characteristics of each image and are later used by machine learning classifiers for texture classification.

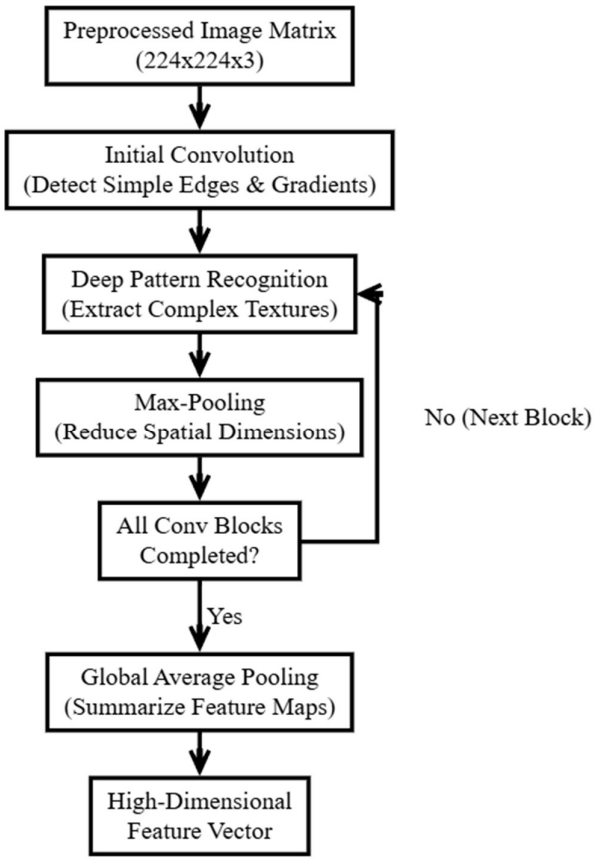


Fig. 4.3: Internal workflow of VGG19.

Input Image Preparation: Fig. 4.3, The preprocessed images are provided as input to the deep convolutional network. Each image is resized to the required dimensions and formed into a numerical array structure. This standardized input ensures compatibility with the feature extraction model.

Convolutional Feature Learning: The input image passes through multiple convolutional layers within the network. These layers apply learnable filters that capture low-level visual patterns such as edges and color variations. As the layers progress, the network begins to identify more complex structures and texture patterns.

Activation and Non-Linear Transformation: After convolution operations, activation functions introduce non-linear transformations to the extracted features. This process enables the network to learn complex visual relationships within the image. The activation layers enhance the network's ability to represent intricate texture variations.

Spatial Down sampling through Pooling: Pooling layers are used to reduce the spatial dimensions of the feature maps generated by convolutional layers. This operation helps

retain important visual information while reducing computational complexity. Pooling also improves the robustness of the feature representation against small image variations.

Feature Vector Generation: After the convolution and pooling operations are completed, global average pooling is applied to convert the feature maps into a compact numerical feature vector. This feature vector summarizes the most important visual information extracted from the image. The generated features are then forwarded to machine learning classifiers for the classification stage.

4.4 Model Building and Training

The machine learning model building stage focuses on training classification algorithms using the feature vectors extracted from the images. After feature extraction, the numerical feature data is divided into training and testing sets to enable effective learning and evaluation of the models. Several machine learning classifiers are implemented to analyse the extracted features and identify patterns that differentiate the texture classes. The classifiers used in this study include LDAC, QDAC, SVC, and ETC. Each model learns from the training data and generates predictions for the testing data to evaluate its classification capability. Performance metrics such as accuracy, precision, recall, and F1-score are used to compare the effectiveness of these models. This model building process helps determine the most reliable classifier for multi-class texture classification.

4.4.1: Linear Discriminant Analysis Classifier (LDAC)

LDAC is a supervised machine learning technique used in this research as one of the baseline classifiers for patch-level texture classification. Its primary goal is to find linear combinations of the extracted VGG19 features that best separate the classes. LDAC reduces feature dimensionality while maximizing class separability, making it efficient for multiclass texture discrimination and enabling faster and more interpretable classification results.

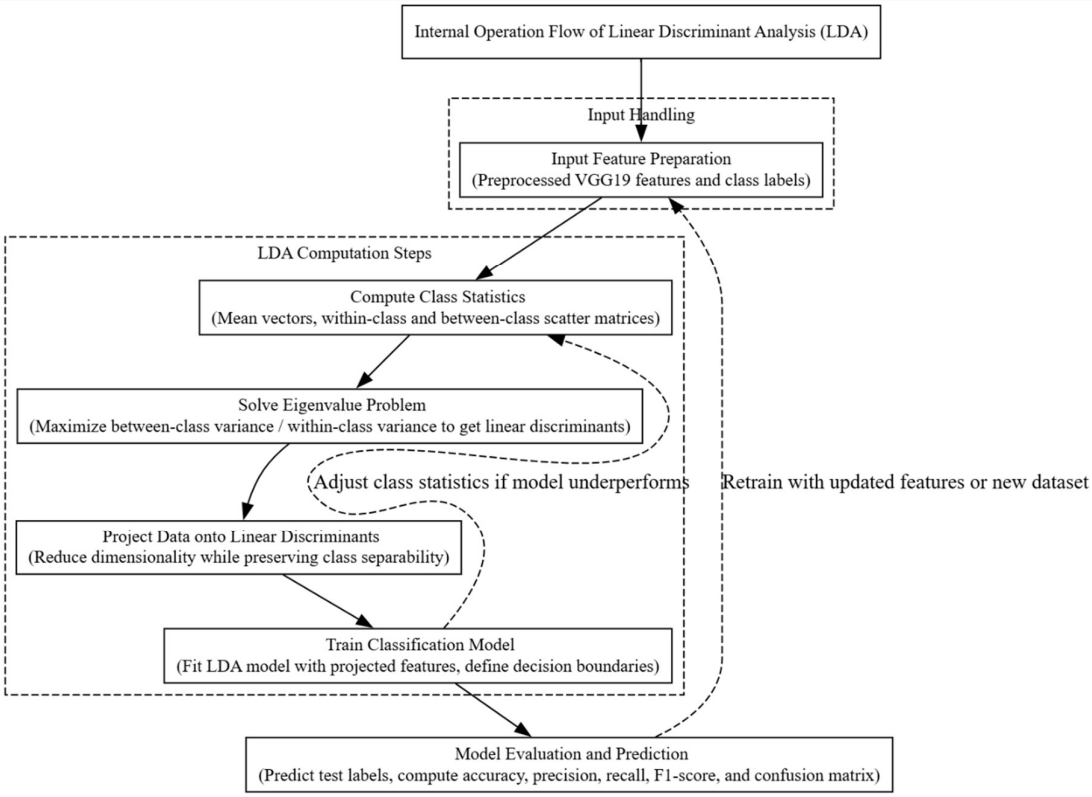


Fig. 4.4: Internal operation flow of LDAC for patch-level texture classification

Input Feature Preparation: Fig. 4.4, The pre-processed VGG19 feature vectors along with their corresponding class labels are taken as input. Each feature vector represents a high-dimensional description of a texture patch. This step ensures that both training and testing data are ready for LDA analysis.

Compute Class Statistics: LDAC computes the mean vector for each class and the overall mean of the dataset. It also calculates the within-class scatter matrix (to measure variance within each class) and the between-class scatter matrix (to measure variance between classes). These matrices capture the spread and orientation of the feature space for effective discrimination.

Solve Eigenvalue Problem: The algorithm solves a generalized eigenvalue problem by maximizing the ratio of between-class variance to within-class variance. This process identifies the optimal linear discriminants (research directions) that best separate the texture classes in a lower-dimensional space.

Research Data onto Linear Discriminants: The original high-dimensional features are researched onto the computed linear discriminant axes. This reduces feature

dimensionality while preserving the class separability. The transformed features now represent the texture patches in a space where classes are maximally distinguishable.

Train Classification Model: The researched training features are used to fit a simple linear classifier. LDA assumes that each class follows a Gaussian distribution with a shared covariance matrix. The model learns to assign class labels based on the researched feature values, forming decision boundaries in the discriminant space.

Model Evaluation and Prediction: Once trained, the LDA model predicts class labels for the test set features. Evaluation metrics such as accuracy, precision, recall, F1-score, and confusion matrix are computed. The results are displayed in the GUI for visual inspection and comparison with other classifiers.

4.4.2. Quadratic Discriminant Analysis Classifier (QDAC)

QDAC is a supervised classification technique like LDAC, but it allows each class to have its own covariance matrix. In this research, QDAC is used as a baseline classifier to separate patch-level texture classes from the TILDA-400 dataset. It models each class with a multivariate Gaussian distribution, capturing more complex class boundaries than LDAC. By fitting quadratic decision surfaces, QDAC can better handle heteroscedasticity (different variances per class) in texture features extracted via VGG19.

Input Feature Preparation: Fig. 4.5, The pre-processed VGG19 feature vectors along with corresponding class labels are prepared. Each feature vector represents a high-dimensional patch-level texture representation. This step ensures that training and testing datasets are ready for QDA processing.

Compute Class Statistics: QDA calculates the mean vector and covariance matrix for each class independently. Unlike LDA, which assumes shared covariance across classes, QDA captures class-specific spread and orientation. This allows it to model elliptical or curved boundaries in the feature space.

Fit Multivariate Gaussian Model: Each class is modelled as a multivariate Gaussian distribution using its mean and covariance. The probability density function of each class is used to determine how likely a feature vector belongs to that class. This probabilistic modelling forms the basis of QDA's decision-making.

Train Quadratic Decision Boundaries: Using the class-specific Gaussian parameters, QDA generates quadratic decision boundaries between classes. These boundaries separate texture classes more flexibly than linear boundaries, accommodating variations in texture patterns.

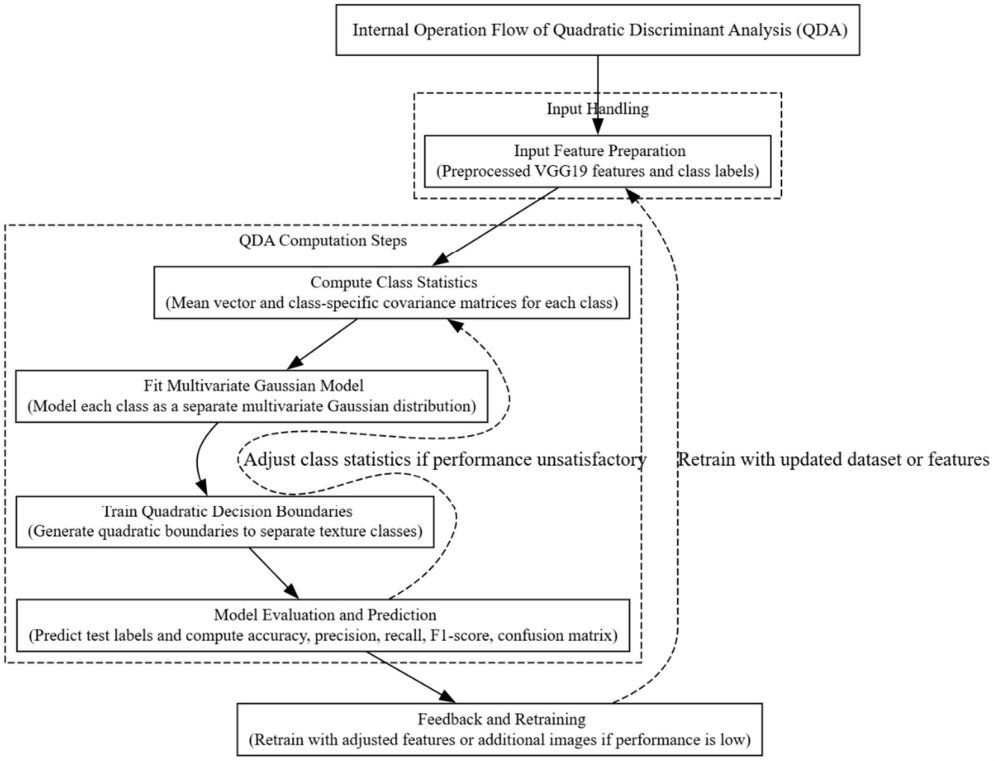


Fig. 4.5: Internal operation flow of QDAC for patch-level texture classification

Model Evaluation and Prediction: The trained QDAC model predicts class labels for the test dataset by evaluating the likelihood of each sample under each class's Gaussian model. Performance metrics such as accuracy, precision, recall, F1-score, and confusion matrix are calculated. The results are displayed in the GUI for visual comparison with other classifiers.

Feedback and Retraining: If the model underperforms, feedback loops allow retraining with adjusted feature sets or hyperparameters. Users can also incorporate additional images to refine class-specific covariance estimation. This ensures that QDAC adapts to variations in texture patches for improved classification.

4.4.4. Support Vector Classifier (SVC)

SVC is a supervised machine learning algorithm used in this research to classify patch-level texture features extracted via VGG19. SVC works by finding the optimal hyperplane that maximally separates classes in the feature space. For non-linear separable textures, the algorithm uses a kernel function (RBF kernel in this case) to map features into a higher-dimensional space, allowing complex decision boundaries. The model's robustness and ability to handle high-dimensional features make it suitable for patch-level texture classification.

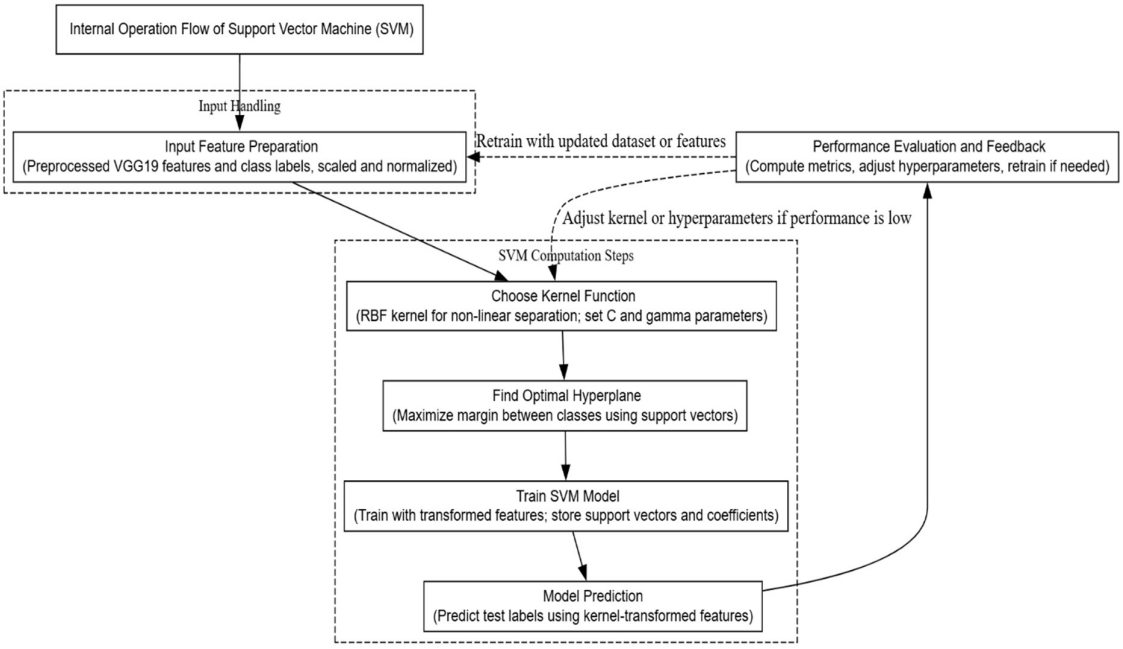


Fig. 4.6: Internal operation of SVC for patch-level texture classification

Input Feature Preparation: The pre-processed VGG19 feature vectors and their corresponding class labels are loaded. Each feature vector encodes texture characteristics of a patch. Proper scaling and normalization ensure stable training for the SVM model.

Choose Kernel Function: The RBF (Radial Basis Function) kernel is selected to handle non-linear separability among texture classes. The kernel transforms features into a higher-dimensional space where classes can be linearly separated. Parameters like C (penalty) and gamma (kernel coefficient) are set to optimize the decision boundary.

Find Optimal Hyperplane: SVC identifies the hyperplane that maximizes the margin between different classes in the feature space. Support vectors, which are the closest points to the hyperplane, determine its position. This ensures that the model generalizes well to unseen texture patches.

Train SVC Model: The model is trained using the training dataset, adjusting weights and biases to satisfy the margin constraints. The RBF kernel allows the hyperplane to curve and adapt to complex texture patterns. The trained model stores the support vectors and coefficients for prediction.

Model Prediction: Test feature vectors are mapped using the same kernel transformation, and the SVC predicts class labels based on their position relative to the hyperplane. Each sample is assigned the class of the region it falls into.

Performance Evaluation and Feedback: Predicted labels are compared with ground truth to calculate accuracy, precision, recall, F1-score, and confusion matrix. If the model underperforms, hyperparameters such as C, gamma, or kernel type can be adjusted, or additional training data can be added. Feedback ensures improved classification performance on texture patches.

4.4.5 Extra Trees Classifier (ETC)

The ETC is an ensemble learning method based on decision trees, used in this research to classify patch-level textures from the TILDA-400 dataset. ETC builds many unpruned decision trees from the training features extracted by VGG19. Unlike traditional Random Forests, it selects split points randomly for each feature, which increases diversity among trees and improves generalization. The final prediction is made by aggregating votes from all individual trees, resulting in robust texture classification even with high-dimensional feature vectors.

Input Feature Preparation: Fig. 4.6, The pre-processed VGG19 feature vectors and their corresponding class labels are loaded. Each feature vector encodes detailed texture information of a patch. This ensures that the ensemble of trees has high-quality input data for training.

Tree Construction with Random Splits: ETC generates multiple decision trees using the training features. At each node of a tree, the split feature and threshold are chosen

randomly rather than optimized. This randomness reduces correlation among trees and improves ensemble robustness.

Node Splitting and Growth: Each tree grows to its full depth without pruning. Nodes are split until they reach a minimum sample count, or all leaves are pure. This step allows each tree to capture complex patterns and interactions between texture features.

Aggregate Predictions from Trees: Once all trees are trained, the ensemble makes predictions by taking a majority vote across individual tree outputs. Each tree casts a vote for the predicted class, and the class with the highest total votes is selected as the final prediction.

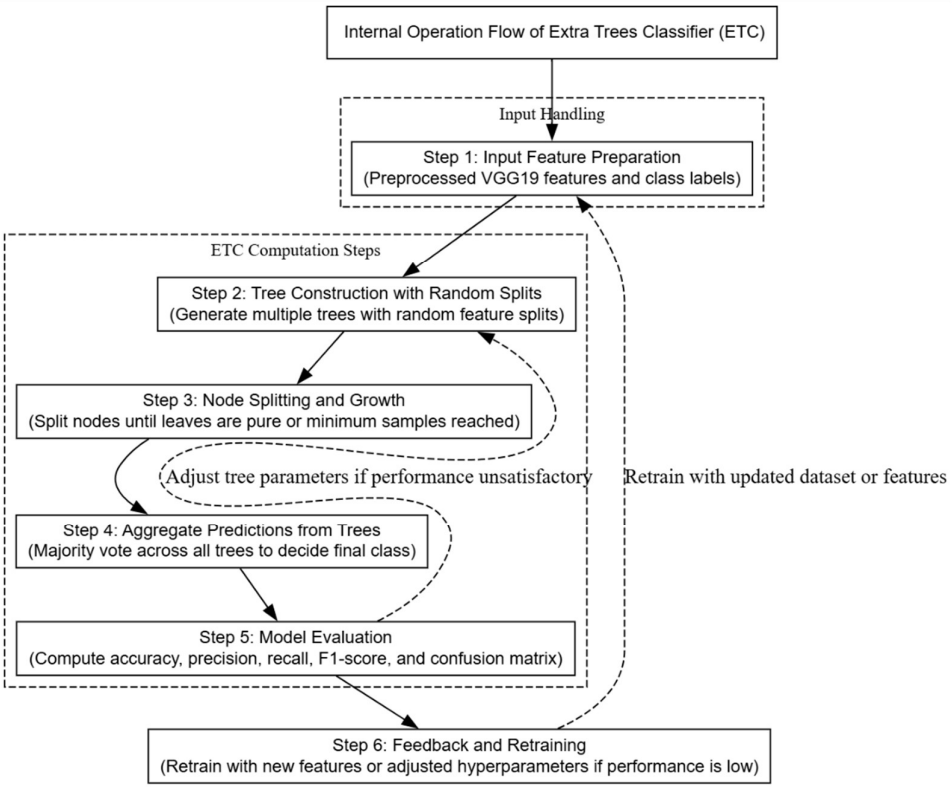


Fig. 4.6: Internal operation flow of ETC for patch-level texture classification

Model Evaluation: The predicted labels for the test dataset are compared against the ground truth to compute metrics like accuracy, precision, recall, F1-score, and confusion matrix. These metrics are displayed in the GUI to evaluate ETC performance relative to other classifiers.

Feedback and Retraining: If the classifier performs sub optimally, users can retrain the model with modified hyperparameters (e.g., number of trees, depth) or additional

texture samples. This feedback mechanism ensures that ETC adapts to new data and maintains high classification accuracy.

CHAPTER 5

UML DIAGRAMS

Unified Modelling Language (UML) is a standardized visual language used to model, design, and document the architecture of software systems. It provides a set of graphical notations to represent the structure and behaviour of a system, making complex systems easier to understand and communicate among developers, stakeholders, and business analysts.

Key Points About UML

Standardized Notation: UML offers a universal set of symbols and diagrams that standardize how software systems are described, which helps teams speak the same language regardless of their background or the programming language they use.

Types of Diagrams: UML includes various diagrams that can be categorized into two main types:

Structural Diagrams: These describe the static aspects of the system. Examples include Class Diagrams, Component Diagrams, and Deployment Diagrams.

Behavioural Diagrams: These focus on the dynamic aspects and interactions within the system. Examples include Use Case Diagrams, Sequence Diagrams, Activity Diagrams, and Collaboration Diagrams.

System Documentation and Communication: UML serves as an effective tool for documenting system requirements, design decisions, and the overall architecture. It helps bridge the gap between technical and non-technical stakeholders by providing clear, visual representations of the system.

Design and Analysis: By modelling different aspects of a system, UML enables developers to analyse and validate the design early in the development process. This can lead to better decision-making, reduced complexity, and improved system maintainability.

Flexibility: UML is versatile and can be used across a wide range of applications, from small-scale research to large, complex systems. It supports object-oriented design principles and can be adapted to various methodologies such as Agile or Waterfall.

5.1 Class Diagram

Fig.5.1 represents the structural components of the system and their relationships. The dataset loader class is responsible for reading images from the dataset folders. The preprocessing class performs image resizing and normalization operations before feature extraction. The feature extractor class uses the VGG19 deep learning model to generate numerical feature vectors from images. These features are then provided to the machine learning models for training and prediction. The classification models include LDAC, QDAC, SVC, and ETC which inherit from a common machine learning model structure. The evaluation class calculates performance metrics such as accuracy, precision, recall, and F1-score. This diagram illustrates the object-oriented structure of the intelligent classification system.

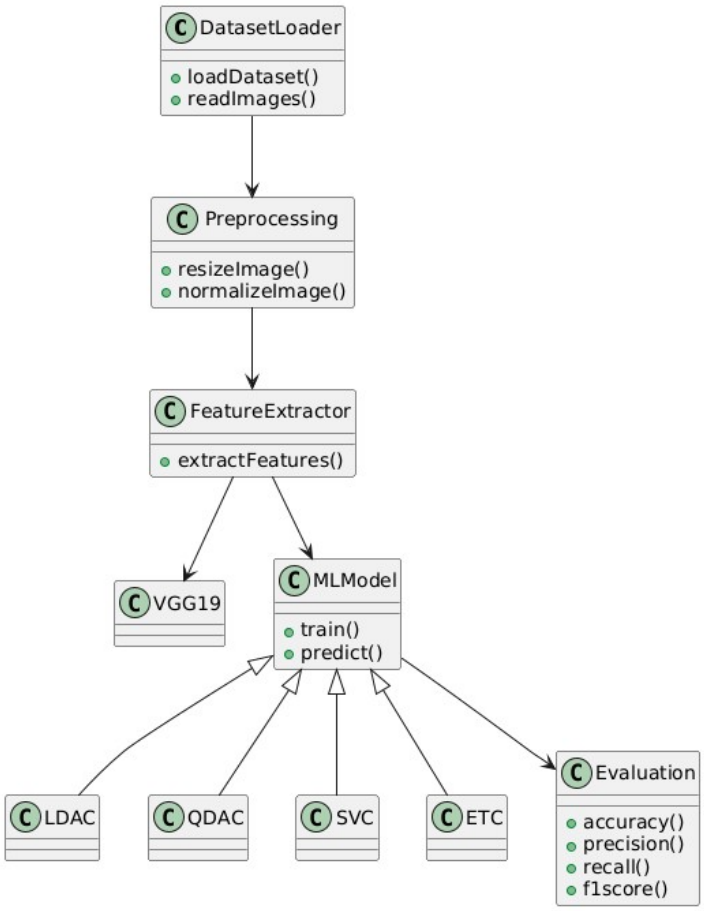


Fig. 5.1: Class diagram for patch-level texture classification

5.2 Use Case Diagram

Fig. 5.2 represents the interaction between users and the system. Two main actors are involved in the system: the administrator and the user. The administrator performs system management tasks such as uploading datasets, running feature extraction, training machine learning models, and evaluating model performance. The user interacts with the system by uploading an image and viewing the predicted class output. The system internally processes the request using deep feature extraction and classification models. The machine learning classifiers such as LDAC, QDAC, SVC, and ETC are used during the model training phase. This diagram illustrates how different users interact with the intelligent image classification system.

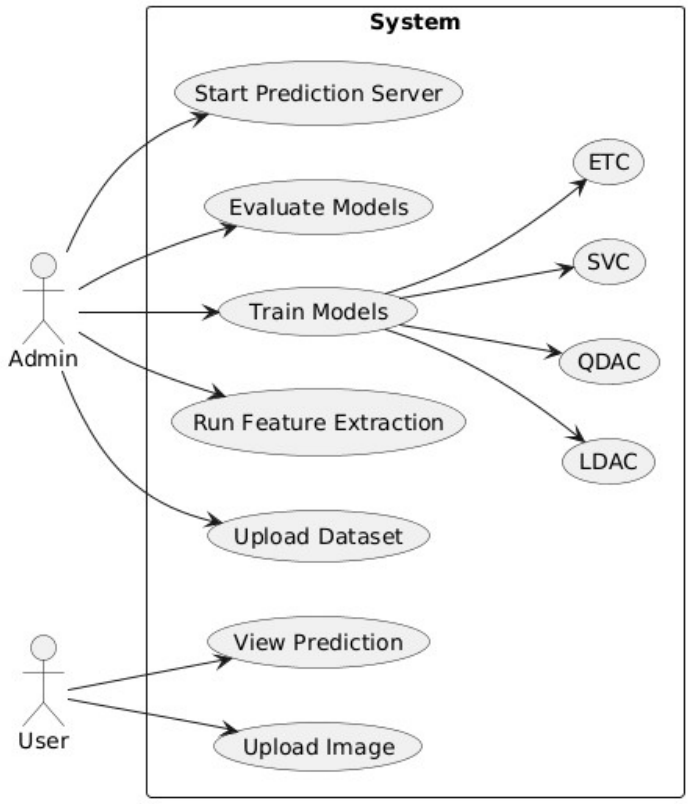


Fig. 5.2: Use case diagram for patch-level texture classification

5.3 Sequence Diagram

Fig. 5.3 illustrates the step-by-step interaction between system components during image classification. The process begins when the user uploads an image through the graphical interface. The image is sent to the preprocessing module where resizing and

normalization operations are performed. The processed image is then forwarded to the VGG19 model to extract deep features. These features are passed to the machine learning classifiers including LDAC, QDAC, SVC, ETC. The classifiers analyse the feature vectors and generate the predicted class label. The prediction result is then returned to the graphical interface and displayed to the user.

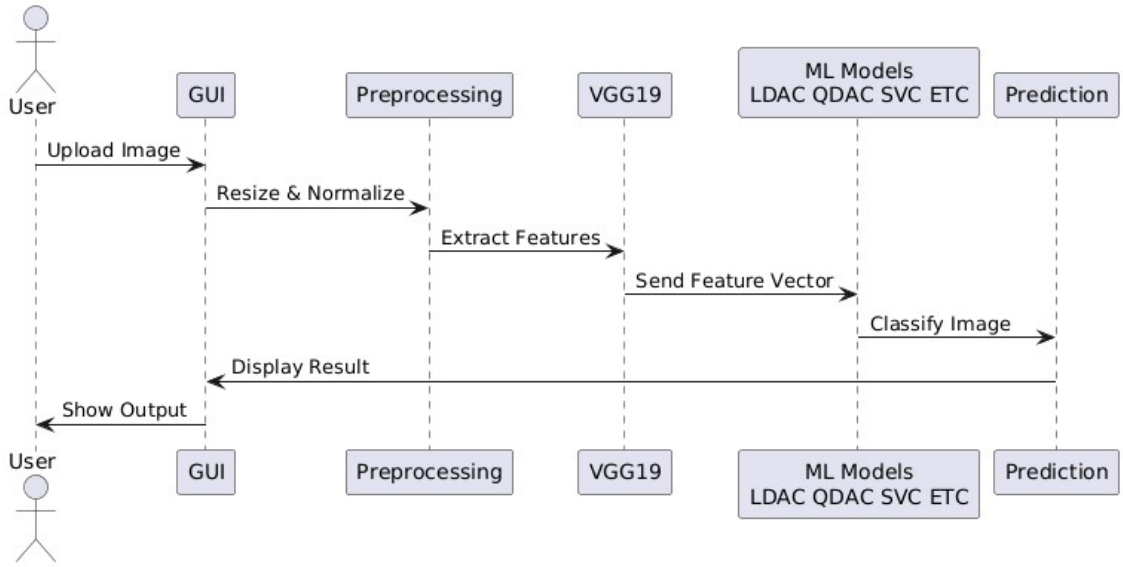


Fig. 5.3: Sequence diagram for patch-level texture classification

5.4 Component Diagram

Fig. 5.4 illustrates the major modules involved in the system and their interactions. The GUI allows users to interact with the application and provide input data. The preprocessing module prepares the images by performing resizing and normalization operations. The feature extraction component uses the VGG19 deep learning model to generate feature vectors from the images. These features are then processed by machine learning classifiers including LDAC, QDAC, SVC, ETC. The evaluation module computes performance metrics to measure classification effectiveness. A Flask-based prediction server enables real-time prediction functionality. This modular architecture improves maintainability and scalability of the system.

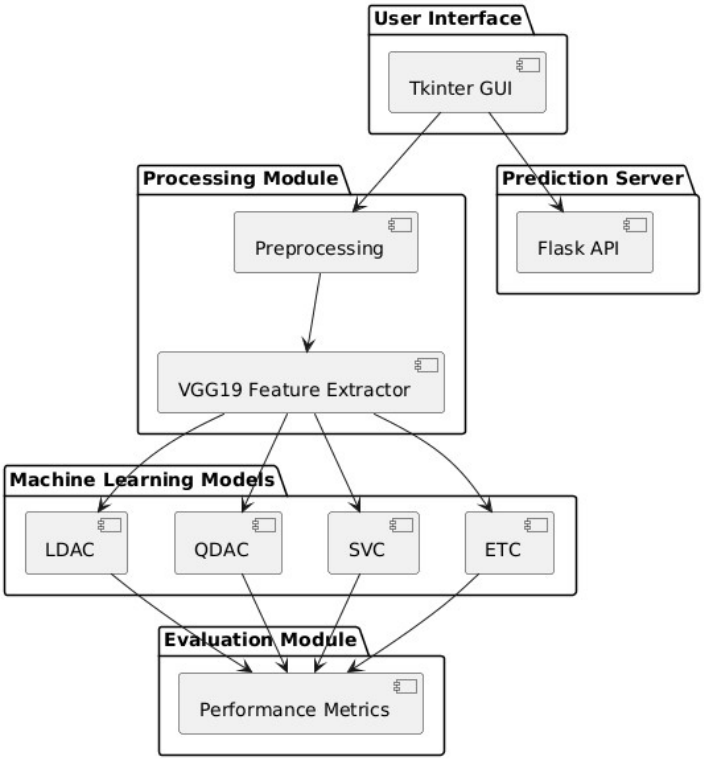


Fig. 5.4: Component diagram for patch-level texture classification

5.5 Deployment Diagram

Fig. 5.5 illustrates how the system components are distributed across hardware nodes. The client system contains the graphical user interface where users interact with the application. The application server runs the Python-based processing modules responsible for preprocessing, feature extraction, and classification. The VGG19 model is deployed on the server to perform deep feature extraction. Machine learning classifiers such as LDAC, QDAC, SVC, and ETC are also hosted on the server for classification tasks. The storage node contains the dataset images and saved trained models. This architecture allows efficient processing of image data and real-time classification operations.

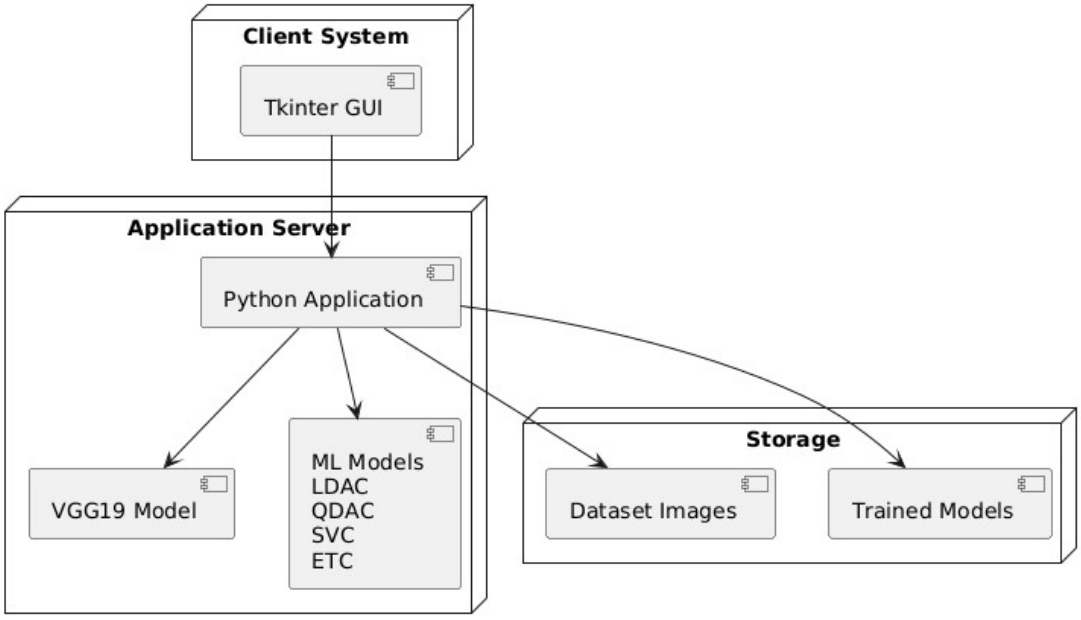


Fig. 5.5: Deployment diagram for patch-level texture classification

5.6 Activity Diagram

Fig. 5.6 represents the workflow of the image classification system. The process begins with dataset upload where industrial texture images are loaded into the system. The images undergo preprocessing operations such as resizing and normalization. Feature extraction is then performed using the VGG19 deep learning model to obtain numerical feature representations. The extracted features are divided into training and testing datasets. Multiple machine learning models including LDAC, QDAC, SVC, and ETC are trained using the training data. The trained models are evaluated using performance metrics to measure their effectiveness. Finally, the best performing model is used to classify new images and generate prediction results.

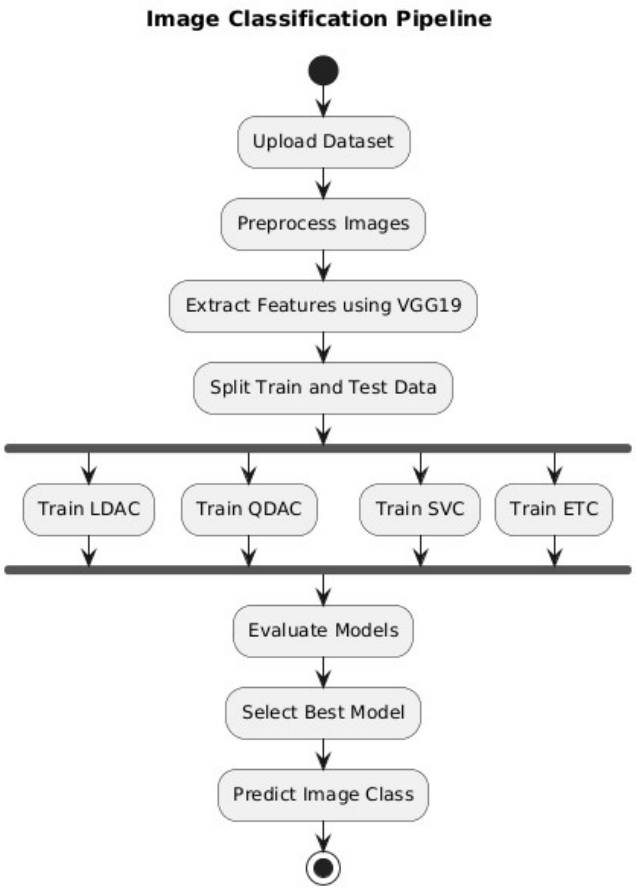


Fig. 5.6: Activity diagram for patch-level texture classification

5.7 Data Flow Diagram (DFD)

Fig. 5.7 illustrates how data moves through the system during image classification. The process begins when the user provides an input image to the system. The image undergoes preprocessing operations such as resizing and normalization to standardize the input data. The processed image is then passed to the VGG19 feature extraction module which generates deep feature representations. These feature vectors serve as input to the machine learning classification models. The classifiers including LDAC, QDAC, SVC, and ETC analyze the features and predict the corresponding class label. The predicted result is finally presented to the user as the classification output.

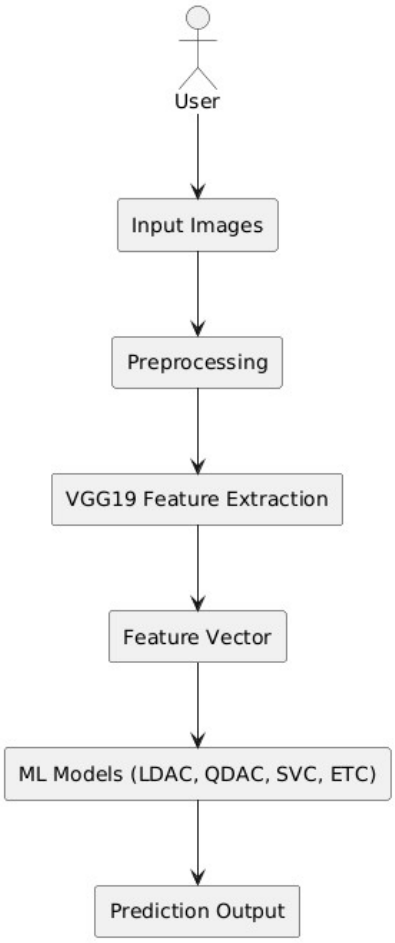


Fig.5.7: Data flow diagram (DFD) for patch-level texture classification

CHAPTER 6

SOFTWARE ENVIRONMENT

6.1 Software Environment

Python 3.12.0 is a maintenance release of Python 3.12, officially released on October 23, 2023. It retains all core features of the 3.12 series with important bug fixes and improvements to enhance stability and performance. Key highlights of Python 3.12 include a built-in breakpoint () function for easier debugging, the new data classes module, which simplifies class creation by automatically generating special methods like __init__, __repr__, and __eq__, and improved support for type hints with better performance and forward references. The insertion-order preservation of dictionaries became an official language feature in this version, making the standard dict replace the need for OrderedDict in many use cases.

Python 3.12.0 also introduced time functions with nanosecond resolution for higher precision, improved customization of module attributes, and significant performance enhancements in annotation handling. It brought improved warnings and handling of legacy locale coercion, helping to avoid common pitfalls related to text encoding. The release includes ongoing refinements to concurrency modules like asyncio, offering better performance and enhanced features. Platform support includes 64-bit and 32-bit Windows, macOS, and major Linux distributions. It requires modest system resources, runs efficiently on contemporary hardware, and integrates with popular development environments.

6.1 Software Requirements

Step 1: Download Python 3.12.0 Installer

- Open your web browser and go to the official Python releases page: https://www.python.org/downloads/release/python-3120/
- Scroll down to the “Files” section.
- Download the Windows installer:
    - For 64-bit Windows, download the Windows x86-64 executable installer.

- For 32-bit Windows, download the Windows x86 executable installer.

Step 2: Run the Installer

- Locate the downloaded .exe file and double-click to run it.
- Important: Before clicking "Install Now," check the box Add Python 3.12 to PATH at the bottom of the installer window. This automatically sets the PATH environment variable for you.
- Click Install Now.

Step 3: Installation Process

- The installer will install Python, pip (Python package manager), and standard libraries.
- Wait for the installation to complete.
- Once done, click Close.

Step 4: Verify Python Installation

- Open Command Prompt (cmd) from the Start menu.

6.1.1 Python Packages

The main Python packages used in the industrial texture classification system are listed in Table 1. These packages collectively support image preprocessing, feature extraction, machine learning model training, evaluation, visualization, and prediction. The selected libraries enable efficient handling of large image datasets and provide reliable learning capabilities for multi-class texture classification. Deep learning frameworks are used to extract meaningful visual features, while machine learning libraries implement classification algorithms for accurate prediction. Visualization tools assist in analysing model performance through graphical representations. The integration of these packages ensures a stable and scalable Python-based environment suitable for intelligent image analysis and automated surface defect detection.

Table 1: Main Python packages used in this research and their key responsibilities

Package	Role in Study	Key Functionalities
Python	Core programming language	Control flow, scripting, integration
NumPy	Numerical computation	Array operations, matrix processing
Pandas	Data handling	Dataset organization, data manipulation
OpenCV	Image processing	Image reading, resizing, image transformation
TensorFlow / Keras	Deep learning framework	VGG19 feature extraction, neural network processing
Scikit-learn	Machine learning framework	LDAC, QDAC, SVC, ETC models, preprocessing, evaluation metrics
Matplotlib	Data visualization	Performance graphs, ROC curves, confusion matrices
Seaborn	Statistical visualization	Heatmaps, distribution plots
Joblib	Model serialization	Saving and loading trained models
Flask	Web framework	API-based prediction service
Tkinter	GUI development	Graphical interface for user interaction
OS	System utilities	File handling, directory management
datetime	Time management	Logging prediction timestamps

Python: Python serves as the primary programming language for implementing the texture classification system. Its simplicity, flexibility, and rich ecosystem of scientific libraries make it highly suitable for machine learning and computer vision research. Python supports seamless integration of deep learning frameworks, image processing libraries, and machine learning models within a unified workflow. The language

enables efficient manipulation of image datasets and simplifies experimental development. Its cross-platform compatibility allows the system to run on various operating systems.

TensorFlow and Keras: TensorFlow and Keras are used for implementing deep learning–based feature extraction in the system. The pretrained VGG19 convolutional neural network is utilized to extract meaningful visual features from industrial texture images. These frameworks provide optimized implementations of convolutional operations and pooling layers required for deep feature learning. They also support efficient numerical computation for large image datasets. The extracted deep features serve as input for machine learning classifiers.

Scikit-learn: Scikit-learn forms the backbone of the machine learning component of the system. It provides implementations of classification algorithms such as LDAC, QDAC, SVC, and ETC that are used for texture classification. The library also offers preprocessing utilities, train–test splitting, and performance evaluation metrics. These tools help measure the effectiveness of the classification models. Scikit-learn's optimized algorithms ensure efficient training and prediction for image feature data.

Pandas and NumPy: Pandas and NumPy are essential for data manipulation and numerical computation within the system. Pandas provides efficient tools for managing datasets, organizing extracted feature vectors, and handling structured data. NumPy supports fast mathematical operations and array processing for feature representation. These libraries form the foundation for preprocessing pipelines and feature vector management. Their efficient memory handling ensures smooth execution when working with large image datasets.

OpenCV: OpenCV is used for image processing operations within the system. It provides functions for loading images, resizing them, and displaying prediction results with visual annotations. OpenCV supports efficient manipulation of image data during preprocessing and prediction stages. The library also enables visualization of classification outputs directly on images. Its optimized image processing capabilities enhance the performance of the system.

Matplotlib and Seaborn: These libraries are used for exploratory data analysis and result visualization. Matplotlib generates visual plots such as confusion matrices, ROC

curves, and performance comparison graphs. Seaborn provides enhanced statistical visualizations with improved aesthetic presentation. Visualization tools help analyze model performance and interpret classification results effectively. These graphical insights assist in understanding the effectiveness of the machine learning models.

Joblib: Joblib is used for serializing and deserializing trained machine learning models. It allows trained classifiers such as LDAC, QDAC, SVC, and ETC to be stored on disk and reused without retraining. This significantly reduces computational overhead during prediction. Joblib also supports efficient handling of large model files. Its fast loading capability ensures smooth model deployment.

Flask and Tkinter: Flask and Tkinter provide the system's interaction and deployment environment. Tkinter is used to build the graphical user interface that allows users to upload datasets, train models, and view results. Flask enables the creation of a lightweight prediction server that processes classification requests. These frameworks allow the system to support both local interaction and API-based prediction services. Their integration enhances the usability and accessibility of the system.

6.2. Hardware Environment

The proposed Python-based image classification system is designed to operate effectively on standard computing platforms, eliminating the need for specialized hardware accelerators. By leveraging a combination of transfer learning-based feature extraction and classical machine learning classifiers, the framework achieves high efficiency on general-purpose processors. While the system is optimized for stability across various configurations, the performance of image preprocessing and model training scales with available hardware resources. This ensures that the methodology remains accessible for standard research environments while being capable of high-speed processing when deployed on recommended hardware setups.

Processor (CPU) Requirements

- The system utilizes multi-core processing for image processing, convolutional feature extraction, and machine learning classification.
- Efficient execution of these operations depends on the clock speed and core count of the central processing unit.

- **Minimum Requirement:** 1.8 GHz dual-core processor.
- **Recommended:** Quad-core processor (Intel Core i5/i7 or AMD Ryzen 5) with 2.5 GHz or higher clock speed.

Memory (RAM) Requirements

- Adequate memory is critical for handling large industrial datasets, which involve loading, preprocessing, and extracting high-dimensional feature vectors.
- Higher RAM capacity directly impacts the speed of model training and the evaluation of complex ensembles.
  - **Minimum Requirement:** 4 GB RAM.
  - **Recommended:** 8 GB RAM or higher to ensure seamless feature extraction and faster processing.

Storage Requirements

- Persistent storage is necessary for maintaining categorized image datasets, pretrained VGG19 model weights, extracted feature files, and generated visualization outputs.
- Sufficient disk space allows for iterative experimentation and the archival of multiple trained model versions.
  - **Minimum Requirement:** 2–3 GB of free disk space.
  - **Recommended:** 15 GB or more to accommodate expanding datasets and comprehensive result logs.

Operating System Compatibility

- The hardware platform must support a 64-bit environment capable of running the latest Python distributions and machine learning libraries.
- The system is fully compatible with:
  - **Windows:** Windows 10 / Windows 11 (64-bit recommended).

- **Cross-Platform:** Major operating systems supporting standard Python environments and backend dependencies.

CHAPTER 7

FUNCTIONAL AND NON-FUNCTIONAL REQUIREMENTS

The industrial texture classification system requires clearly defined system requirements to ensure accurate image analysis, efficient feature extraction, and reliable classification performance. Well-structured functional and non-functional requirements guide the development process and ensure that each component of the system operates correctly. These requirements help the system process large volumes of image data, perform deep feature extraction, and apply multiple machine learning classifiers effectively. They also ensure that performance, scalability, and reliability are maintained during model training and prediction. In addition, a system study evaluates the feasibility and practicality of implementing the intelligent image classification framework in real-world industrial inspection environments.

7.1 Functional Requirements

Functional requirements define the core operations and services that the texture classification system must perform to achieve its objectives. These requirements describe how image data is processed, how users interact with the system, and how classification results are generated. They ensure that all stages, from image preprocessing to model evaluation, operate consistently and accurately. For this study, functional requirements focus on dataset handling, feature extraction, machine learning model training, and prediction processes. Clearly defined functional requirements help the system produce reliable classification results and ensure smooth coordination between different processing modules.

Dataset Input and Management: The system must allow users to upload an image dataset organized into folders representing different texture classes. The dataset should be validated to ensure that images are correctly structured and readable by the system. Proper dataset handling ensures smooth execution of preprocessing and feature extraction stages.

Image Preprocessing Operations: The system must perform preprocessing operations such as resizing images to a standard resolution and converting them into numerical

arrays. Normalization techniques should be applied to improve image quality and compatibility with deep learning models. Effective preprocessing ensures consistent input data for feature extraction.

Feature Extraction: The system must extract meaningful visual features from images using the VGG19 transfer learning model. The pretrained convolutional neural network processes the images and generates feature vectors representing texture patterns. These feature vectors serve as input for machine learning classifiers.

Model Training and Prediction: The system must train multiple machine learning classifiers including LDAC, QDAC, SVC, and ETC. Each model should analyze the extracted features and generate predictions for texture classification.

Model Evaluation and Comparison: The system must evaluate the trained models using performance metrics such as accuracy, precision, recall, F1-score, confusion matrix, and ROC curves. Evaluation results should be displayed clearly to compare the effectiveness of different classifiers.

Prediction Result Visualization: The system should display classification results directly on the processed image along with the predicted class label. Visual outputs help users easily interpret classification outcomes. The system should also provide graphical performance visualizations.

Model Storage and Loading: The system should store trained models using serialization techniques so that they can be reused during prediction without retraining. This improves system efficiency and supports consistent prediction of workflows.

7.2 Non-Functional Requirements

Non-functional requirements describe the quality attributes that define how the texture classification system performs rather than what it does. These requirements ensure the system remains efficient, reliable, and scalable while handling large volumes of image data. They guide architectural and implementation decisions to improve robustness and usability. Non-functional requirements also ensure consistent performance when the dataset size increases or when additional classifiers are integrated. By addressing these attributes, the system maintains long-term stability and reliability in industrial inspection applications.

Performance Requirements: The system should process images efficiently and generate classification results within acceptable time limits. Feature extraction and classification operations must be optimized to reduce computational delay. Efficient processing improves usability during experimentation and prediction.

Reliability Requirements: The system must ensure consistent preprocessing, feature extraction, and classification of operations during execution. Data corruption or loss should be avoided while processing image datasets. Reliable performance ensures trustworthy classification results.

Scalability Requirements: The system should be capable of handling increasing numbers of images without significant performance degradation. It should also support the addition of new machine learning models or datasets in the future.

Security Requirements: The system should ensure safe handling of image datasets and trained models. Access to administrative operations such as dataset upload and model training should be restricted to authorized users. Proper security maintains data integrity.

Usability Requirements: The system interface should be simple and easy to understand for users with different technical backgrounds. Classification results and visual outputs should be clearly displayed to improve interpretability.

Maintainability Requirements: The system should follow a modular architecture where preprocessing, feature extraction, and classification modules operate independently. This design allows easy modification or replacement of individual components without affecting the entire system.

7.3 System Study

A comprehensive system study is essential when developing an intelligent texture classification system due to the complexity of image data and the need for accurate defect detection. This study involves analysing existing industrial inspection methods, identifying limitations in manual inspection, and evaluating the feasibility of automated image analysis solutions. By examining current approaches, the study highlights challenges such as large image datasets, complex texture variations, and the need for reliable classification models. It provides a foundation for designing a robust automated

inspection system. The system study ensures that the research objectives align with practical industrial requirements.

Problem Analysis: Traditional surface inspection methods rely heavily on manual observation, which can lead to inconsistent results and human error. Many conventional image processing approaches struggle to detect subtle defects or classify complex textures accurately. These limitations highlight the need for intelligent image analysis techniques. Automated classification systems can significantly improve inspection efficiency.

Objectives and Requirements Gathering: System requirements were identified through analysis of industrial inspection needs and image classification research studies. The system must support deep feature extraction, multiple machine learning classifiers, and performance evaluation mechanisms. Requirements also include visualization and prediction capabilities.

Feasibility Study: Technical feasibility was confirmed through the use of Python-based machine learning libraries such as TensorFlow, Keras, Scikit-learn, and OpenCV. Economic feasibility is achieved by using open-source software tools. Operational feasibility ensures that the system can be easily used through a graphical interface.

System Analysis and Modelling: Various UML diagrams including class diagrams, use case diagrams, sequence diagrams, activity diagrams, component diagrams, and deployment diagrams are used to represent system structure and workflow. These diagrams help visualize data flow and interactions between modules. System modelling simplifies complex processes and supports better implementation planning.

Architectural Design: The system follows a modular architecture consisting of preprocessing, feature extraction, machine learning classification, evaluation, and prediction modules. The feature extraction component uses the VGG19 deep learning model, while classification is performed using LDAC, QDAC, SVC, and ETC models. This architecture improves scalability, maintainability, and system performance.

Conclusion: The system study confirms that an automated image classification framework based on deep feature extraction and machine learning classifiers is feasible and effective for industrial texture analysis. The integration of preprocessing, transfer learning, and classification models enables accurate defect detection. The study establishes a strong foundation for implementing an intelligent inspection system.

CHAPTER 8

SOURCE CODE

```
#  Core Libraries

import os, cv2, joblib

import numpy as np

import seaborn as sns

import matplotlib.pyplot as plt

import tkinter as tk

from tkinter import filedialog, Text, Scrollbar, Label, Button, END

from PIL import Image, ImageTk


#Machine Learning

from sklearn.model_selection import train_test_split

from sklearn.metrics import (accuracy_score, precision_score, recall_score,

                             f1_score, confusion_matrix, classification_report)

from sklearn.discriminant_analysis import LinearDiscriminantAnalysis,
QuadraticDiscriminantAnalysis

from sklearn.ensemble import ExtraTreesClassifier

from sklearn.svm import SVC


# Deep Learning

from keras.preprocessing import image

from tensorflow.keras.applications.vgg19 import VGG19, preprocess_input
```

```
# GLOBALS
X, Y        = [], []
filename    = ""
categories  = []
model_folder = "model"

base_model = VGG19(weights='imagenet', include_top=False,
                   pooling='avg', input_shape=(224, 224, 3))


# MODULE 1 — DATASET LOADING
def uploadDataset():
    global filename, categories
    filename   = filedialog.askdirectory(initialdir=".")
    categories = [d for d in os.listdir(filename)
                  if os.path.isdir(os.path.join(filename, d))]
    log(f"Dataset loaded\nClasses: {categories}")

# MODULE 2 — VGG19 FEATURE EXTRACTION
def VGG19_feature_extraction():
    global X, Y
    data_path  = "model/X_compressed.npz"
    label_path = "model/Y.npy"
```

```
    if os.path.exists(data_path) and os.path.exists(label_path):
        X = np.load(data_path)['X']
        Y = np.load(label_path)
    else:
        X, Y = [], []
        for label, name in enumerate(categories):
            for img_file in os.listdir(os.path.join(filename, name)):
                if img_file.lower().endswith(('.jpg', '.jpeg', '.png', '.bmp')):
                    img   = image.load_img(os.path.join(filename, name, img_file),
target_size=(224, 224))
                    x     = preprocess_input(np.expand_dims(image.img_to_array(img),
axis=0))
                    feats = base_model.predict(x, verbose=0).squeeze()
                    X.append(feats); Y.append(label)
        X, Y = np.array(X), np.array(Y)
        np.savez_compressed(data_path, X=X); np.save(label_path, Y)

    log(f"VGG19 Feature Extraction complete\nFeature shape: {X.shape}")

# MODULE 3 — TRAIN / TEST SPLIT
def Train_test_spliting():
    global X_train, X_test, y_train, y_test
    idx_file = os.path.join(model_folder, "shuffled_indices.npy")
    idx = np.load(idx_file) if os.path.exists(idx_file) \
```

```
        else np.random.permutation(X.shape[0])
    np.save(idx_file, idx)
    Xs, Ys = X[idx], Y[idx]
    X_train, X_test, y_train, y_test = train_test_split(Xs, Ys, test_size=0.2,
random_state=42)
    log(f"Train: {X_train.shape} | Test: {X_test.shape}")

# MODULE 4 — PERFORMANCE EVALUATION
def performance_evaluation(label, algorithm, predict, y_test):
    a = accuracy_score(y_test, predict) * 100
    p = precision_score(y_test, predict, average='macro') * 100
    r = recall_score(y_test, predict, average='macro') * 100
    f = f1_score(y_test, predict, average='macro') * 100

    cm = confusion_matrix(y_test, predict)
    se = cm[0, 0] / (cm[0, 0] + cm[0, 1]) * 100
    sp = cm[1, 1] / (cm[1, 0] + cm[1, 1]) * 100

    log(f"{algorithm}\n  Accuracy:{a:.2f}  Precision:{p:.2f}  Recall:{r:.2f}
F1:{f:.2f}")
    log(f"  Sensitivity:{se:.2f}  Specificity:{sp:.2f}")
    log(classification_report(y_test, predict, target_names=categories))

    plt.figure(figsize=(6, 6))
    sns.heatmap(cm, xticklabels=categories, yticklabels=categories,
```

```
            annot=True, fmt='g', cmap='viridis')
    plt.title(f"{algorithm} Confusion Matrix")
    plt.ylabel('True class'); plt.xlabel('Predicted class'); plt.show()
    return a, p, r, f


# MODULE 5 — CLASSIFIER MODELS
def _load_or_train(pkl_name, clf):
    path = os.path.join(model_folder, pkl_name)
    if os.path.exists(path):
        return joblib.load(path)
    clf.fit(X_train, y_train); joblib.dump(clf, path); return clf


def Model_LDA():
    m = _load_or_train("LDA_model.pkl", LinearDiscriminantAnalysis())
    performance_evaluation(categories, "Existing LDA", m.predict(X_test), y_test)


def Model_QDA():
    m = _load_or_train("QDA_model.pkl", QuadraticDiscriminantAnalysis())
    performance_evaluation(categories, "Existing QDA", m.predict(X_test), y_test)


def Model_SVM():
    m = _load_or_train("SVM_model.pkl",
        SVC(kernel='rbf', C=100, gamma=0.001, class_weight='balanced',
probability=True))
    performance_evaluation(categories, "Existing SVM", m.predict(X_test), y_test)
```

```
def ETC_model():
    m = _load_or_train("ETC_model.pkl",
        ExtraTreesClassifier(n_estimators=200, max_depth=32,
                             max_features='sqrt', random_state=42))
    performance_evaluation(categories, "Proposed VGG19+ETC", m.predict(X_test),
y_test)


# MODULE 6 — SINGLE IMAGE PREDICTION
def predict():
    clf  = joblib.load(os.path.join(model_folder, "ETC_model.pkl"))
    path = filedialog.askopenfilename(initialdir="testImages")
    x    = preprocess_input(np.expand_dims(
              image.img_to_array(image.load_img(path, target_size=(224, 224))),
axis=0))
    pred  = int(clf.predict(base_model.predict(x).reshape(1, -1))[0])
    label = categories[pred]
    frame = cv2.resize(cv2.imread(path), (800, 800))
    cv2.putText(frame, f'Classified as: {label}', (10, 25),
                cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)
    cv2.imshow(f'Classified as: {label}', frame); cv2.waitKey(0)
# GUI LAYOUT
def log(msg):
    text.insert(END, str(msg) + "\n")
```

```
main = tk.Tk()
main.title("TILDA"); main.geometry("1300x1200")

bg_photo = ImageTk.PhotoImage(
   Image.open("background.jpg").resize((1380, 730), Image.LANCZOS))
tk.Label(main, image=bg_photo).place(relwidth=1, relheight=1)

Label(main,
     text="Patch-Level Texture Classification in Remote Sensing Using Transfer Learning on TILDA-400",
     font=('Times', 15, 'bold')).place(x=300, y=15)

ff = ('Times', 12, 'bold')
buttons = [
   ("Upload Dataset",          uploadDataset,          100),
   ("VGG19 Feature Extraction", VGG19_feature_extraction, 150),
   ("Train Test Splitting",    Train_test_spliting,    200),
   ("Existing LDA",            Model_LDA,              250),
   ("Existing QDA",            Model_QDA,              300),
   ("Train SVM",               Model_SVM,              350),
   ("Train ETC",               ETC_model,              400),
   ("Upload Test Image",       predict,                450),
   ("Exit",                    main.destroy,           500),
]
for lbl, cmd, y in buttons:
```

```
    Button(main, text=lbl, command=cmd, bg="lightblue", font=ff).place(x=20, y=y)


text   = Text(main, height=22, width=70, font=ff)

scroll = Scrollbar(main, command=text.yview)

text.configure(yscrollcommand=scroll.set)

text.place(x=450, y=100)

scroll.place(x=1005, y=100, height=352)


main.config(bg='SkyBlue')

main.mainloop()
```

RESULTS AND DISCUSSION

9.1 Implementation description

The implementation of the texture classification framework is carried out through a structured pipeline of predefined and inbuilt Python functions. The workflow ensures that image data flows smoothly from raw dataset input to feature extraction, machine learning model training, and final prediction. Deep learning–based feature extraction and classical machine learning classifiers are integrated to improve classification accuracy. The system also includes a graphical user interface and a lightweight prediction server for user interaction and remote inference. The following steps describe the internal operational flow of the implemented framework.

1. Environment & GUI Initialization

- The system initializes the user interface and prepares the backend data structures.
- **tkinter.Tk() & main.mainloop():** Establish the primary graphical window and the event-driven loop.
- **ImageTk.PhotoImage():** Renders the visual elements and background of the interface.
- **TinyDB() & hashlib.sha256():** Manage local storage for user credentials and ensure secure, hashed authentication.

2. Dataset Loading & Image Ingestion

- This module handles the ingestion of raw visual data into the processing pipeline.
- **filedialog.askdirectory():** Provides a native OS folder picker for selecting the image dataset.
- **os.listdir() & os.path.isdir():** Programmatically map the folder structure to identify class labels.

- **image.load_img() & image.img_to_array():** Convert physical image files into numerical tensors for deep learning compatibility.

3. Image Preprocessing

- Raw images are standardized to ensure consistent feature extraction.
- **target_size=(224,224):** Resizes all inputs to the specific input dimensions required by the VGG19 architecture.
- **preprocess_input():** Normalizes pixel values to match the distribution of the ImageNet dataset.
- **np.savez_compressed():** Optimizes storage by saving extracted features in a compressed format for rapid retrieval.

4. Deep Feature Extraction (VGG19)

- Leveraging transfer learning, the system extracts high-level spatial features.
- **VGG19(include_top=False):** Loads the pretrained model without the classification head to function as a feature descriptor.
- **base_model.predict():** Generates multi-dimensional feature maps from the input images.
- **np.squeeze():** Flattens the output into a 1D numerical vector suitable for classical machine learning input.

5. Dataset Splitting & Preparation

- Prepares the extracted features for the training phase.
- **np.random.shuffle():** Randomizes the data to eliminate any collection bias.
- **train_test_split():** Segregates the data into training (for learning) and testing (for validation) subsets.

6. Machine Learning Model Training

- The system trains both baseline and proposed models using the extracted VGG19 features.

- **LinearDiscriminantAnalysis() & QuadraticDiscriminantAnalysis():** Statistical baselines for class separation.
- **SVC(kernel='rbf'):** A non-linear classifier used to handle complex feature boundaries.
- **ExtraTreesClassifier():** The proposed ensemble model that aggregates multiple randomized decision trees.
- **joblib.dump():** Serializes the trained .pkl files for persistent storage and future inference.

7. Performance Evaluation & Visualization

- Post-training analysis to verify model reliability.
- **accuracy_score(), precision_score(), recall_score(), f1_score():** Quantitative metrics to assess classification success.
- **confusion_matrix() & roc_curve():** Visual indicators of model performance across different texture classes.
- **sns.heatmap():** Renders the statistical results into intuitive visual charts within the GUI.

8. Prediction & Inference

- The deployment phase where the system classifies new, unseen samples.
- **filedialog.askopenfilename():** Allows users to select a specific image for real-time analysis.
- **cv2.putText() & cv2.imshow():** Uses OpenCV to overlay the predicted label (e.g., "Hole" or "Normal") directly onto the visual output.

9. API-Based Prediction Server

- Enables the framework to act as a backend service for external applications.
- **Flask():** Initializes a lightweight web server.
- **@app.route('/predict'):** Defines the endpoint for receiving external image files.

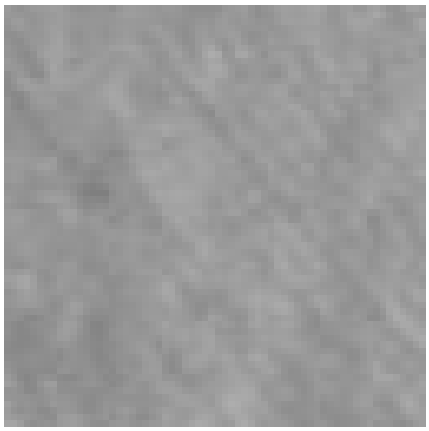
- **threading.Thread():** Ensures the API server runs concurrently with the GUI, preventing the interface from freezing during remote requests.

9.2 Dataset Description

The dataset used in this research is designed for patch-level texture classification and consists of three distinct classes: Good, Hole, and Objects. The Good class represents defect-free surface patches with uniform textures, serving as the baseline for comparison with defective samples. The Hole class contains images where the surface has missing material or visible holes, which disrupt the natural texture patterns and are critical to detect in quality inspection processes as shown in table 2. The Objects class includes images of patches containing unwanted foreign objects or external contaminants that introduce irregularities in the surface texture. The dataset is organized into class-specific folders, making it easy to label and process, and all images are standardized to a resolution of 224 × 224 pixels to ensure compatibility with the VGG19 feature extractor while preserving essential texture details. The dataset is balanced across the three categories to avoid bias in classification and provides sufficient variation in lighting and orientation, which enhances robustness and generalization of the trained model. By combining both defect-free and defective samples, this dataset provides a strong foundation for building an effective vision transformer-based framework for texture classification in remote sensing.

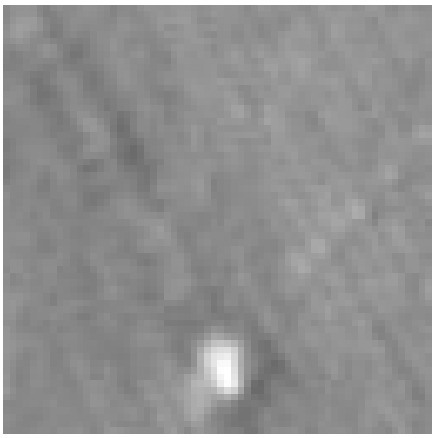
Table. 2: Dataset summary for patch-level texture classification

Class	**Description**	**Number of Images**
Good	Normal, defect-free surface patches	1001
Hole	Patches containing visible holes or missing material	1101
Objects	Patches containing unwanted foreign objects or contaminants	1049



Screenshot. 9.1: Sample image from the good class

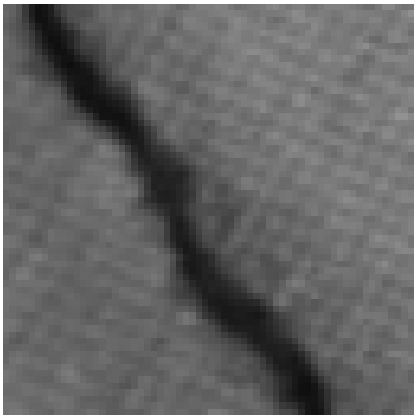
Screenshot.9.1 talks about a good class in the dataset representing defect-free surface patches that exhibit uniform and consistent texture patterns. These images serve as the baseline reference for comparison with defective samples, helping the model learn the distinguishing features of normal textures. Accurate identification of this class is crucial, as it ensures that defect-free patches are not incorrectly flagged as defective, thereby reducing false alarms in automated inspection systems. The inclusion of diverse good samples, captured under varying lighting and orientations, enhances the robustness of the dataset and enables the model to generalize effectively to real-world conditions.



Screenshot. 9.2: Sample image from the hole class

Screenshot. 9.2 talks about hole class contains surface patches with visible holes or missing regions of material that disrupt the natural texture. Such defects are critical to detect because they compromise the structural integrity and overall quality of the

surface. The irregular patterns formed by these holes make them distinct from defect-free textures, providing clear visual cues for classification. Correct recognition of this class is essential to ensure that damaged materials are not passed as acceptable, and the variety of hole patterns in the dataset enables the model to handle different defect shapes and sizes effectively.



Screenshot. 9.3: Sample image from the objects class

Screenshot. 9.3 talks about objects class that includes surface patches with unwanted foreign materials or contaminants present on them. These defects appear as irregular patterns, spots, or intrusions that do not belong to the natural texture of the surface. Unlike holes, which indicate material loss, object-based defects are caused by external interference, making them equally important to identify for quality assurance. Accurately classifying this class helps in detecting contamination or anomalies that could affect downstream processes. The dataset provides sufficient variations of object defects, enabling the model to generalize across diverse external disturbances during classification.

9.3 Result Description

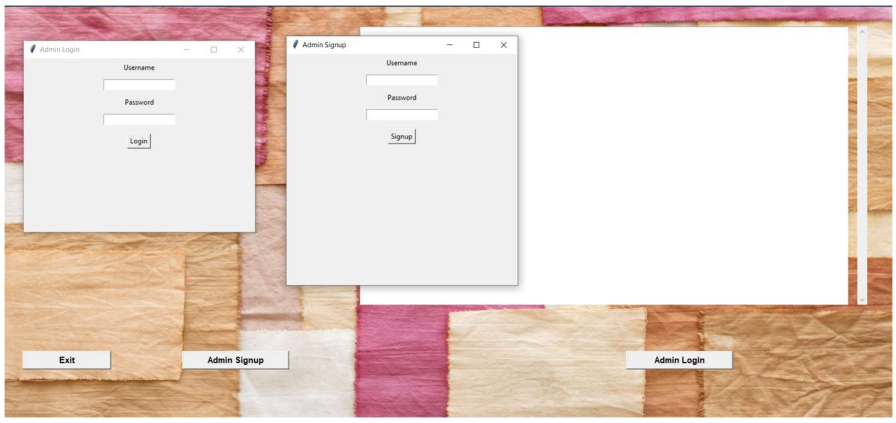
The results of the study demonstrate the effectiveness of the proposed texture classification framework in analysing industrial surface images. The system processes the input dataset through preprocessing, deep feature extraction, and machine learning classification stages to generate accurate predictions. Experimental results are obtained by training multiple classifiers and evaluating their performance using standard evaluation metrics. The results provide insights into how effectively the classifiers distinguish between different texture categories such as normal surfaces, holes, and

foreign objects. Performance comparisons among the implemented models highlight the strengths and limitations of each classifier. These outcomes help determine the most reliable model for accurate industrial texture classification.



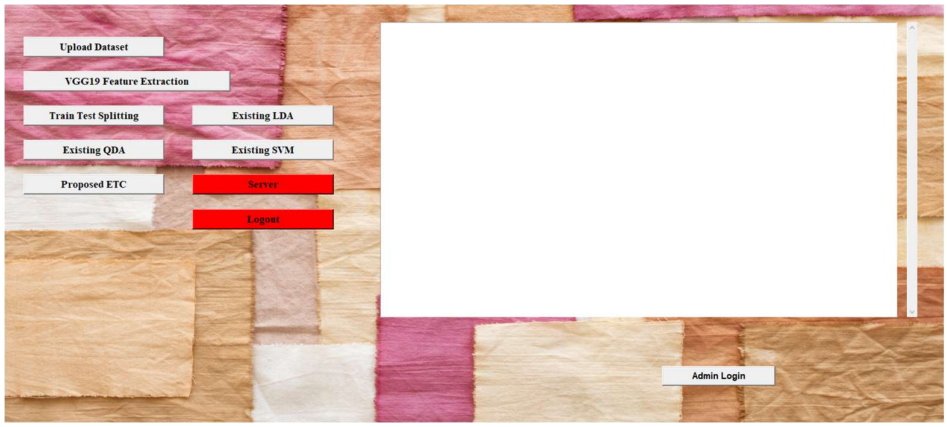
Screenshot. 9.4: GUI of the Texture Classification System

Screenshot. 9.4 illustrates the graphical user interface of the implemented texture classification system designed for analyzing industrial image datasets. The interface acts as the primary interaction layer through which users access various functionalities of the framework. It allows administrators to perform operations such as authentication, dataset management, and system initialization before executing the classification workflow. The interface serves as a control environment where users can trigger processes including feature extraction, machine learning model training, evaluation, and prediction. It also provides an area where system outputs, logs, and processing results can be displayed during execution. This interface ensures smooth coordination between the user and the backend modules responsible for image preprocessing, VGG19 feature extraction, and machine learning classification using LDAC, QDAC, SVC, and ETC models.



Screenshot. 9.5: Administrator Authentication Interface (Login and Signup)

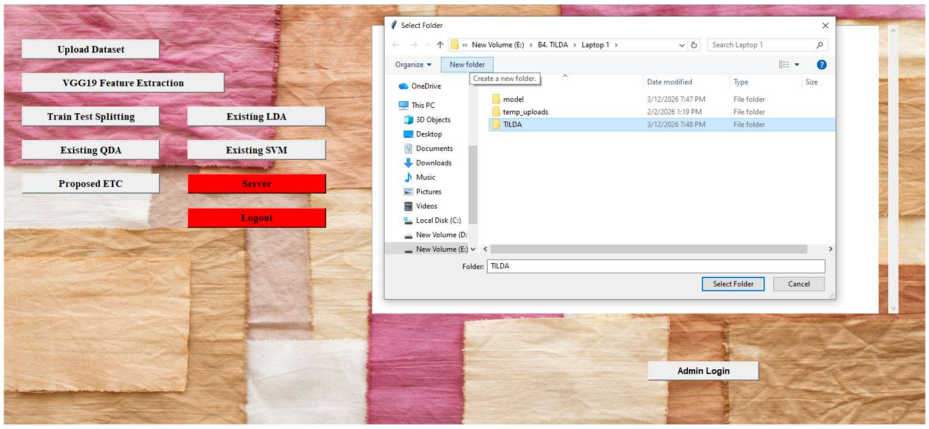
Screenshot. 9.5 depicts the administrator authentication interface used to control access to the texture classification framework. This interface provides secure login and registration mechanisms that allow administrators to create credentials and authenticate themselves before accessing system functionalities. The authentication process ensures that only authorized users can perform administrative operations such as dataset uploading, model training, and system management. User credentials entered through this interface are validated and securely stored using hashing techniques to protect sensitive information. The login module verifies the stored credentials before granting access to the system dashboard and processing modules. This authentication layer enhances the security and reliability of the overall framework by preventing unauthorized access to the machine learning and image processing components.



Screenshot. 9.6: Administrative Control Dashboard for System Operations

Screenshot. 9.6 illustrates the administrative control dashboard of the texture classification system where all major processing operations are initiated. This interface

provides centralized access to the different stages of the analysis pipeline, allowing the administrator to manage dataset loading, feature extraction, model training, and evaluation procedures. Through this control panel, the system executes deep feature extraction using the VGG19 model and performs dataset partitioning for training and testing processes. The interface also enables the execution of machine learning classifiers including LDAC, QDAC, SVC, and the proposed ETC model for texture classification. Each operation can be triggered sequentially to ensure a structured workflow from data preparation to model evaluation. The dashboard supports server activation and system session management, enabling efficient monitoring and control of the classification framework.



Screenshot. 9.7: Dataset Selection and Upload Interface

Screenshot. 9.7 illustrates the dataset upload operation within the texture classification system. This interface enables the administrator to select the dataset directory containing categorized texture images required for the analysis process. The dataset selection mechanism allows the system to access image folders corresponding to different texture classes, which serve as the input for preprocessing and feature extraction stages. Once the dataset is selected, the system prepares the images for further processing by identifying class labels and organizing the image data for training and testing operations. This step forms the initial stage of the analysis pipeline and ensures that the system receives structured input data for deep feature extraction using the VGG19 model. Proper dataset loading is essential for enabling the subsequent machine learning classifiers such as LDAC, QDAC, SVC, and ETC to learn and perform accurate texture classification.



Screenshot. 9.8: VGG19 Feature Extraction Output Display

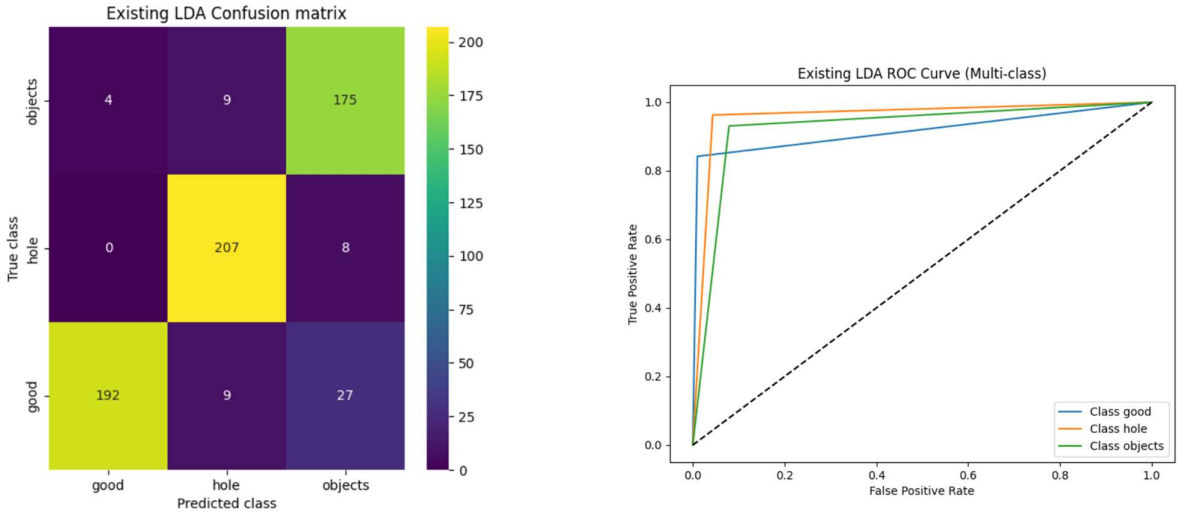
Screenshot. 9.8 illustrates the feature extraction stage of the texture classification system where deep features are generated from the uploaded image dataset. This stage represents the completion of image preprocessing followed by the extraction of high-level visual features using the pretrained VGG19 convolutional neural network. During this process, each image is passed through the convolutional layers of the network to capture important texture patterns and structural characteristics. The extracted features are transformed into numerical feature vectors that represent the visual information contained in the images. The output displayed in this interface confirms that preprocessing and feature extraction have been successfully completed. These generated feature vectors serve as the input data for the subsequent machine learning classifiers including LDAC, QDAC, SVC, and ETC for performing multi-class texture classification.

Screenshot. 9.8 illustrates the dataset partitioning stage of the texture classification system where the extracted feature vectors are divided into training and testing datasets. This step is performed after the completion of feature extraction using the VGG19 model and prepares the data for machine learning model training and evaluation. The system organizes the feature vectors and corresponding class labels into structured subsets that allow the classifiers to learn patterns from the training data and validate performance using the testing data. The displayed information shows the dimensional structure of the input feature data along with the size of the training and testing subsets. This partitioning process ensures that the classifiers are trained on one portion of the dataset while their predictive performance is evaluated on unseen data. The prepared

datasets are then used by machine learning classifiers such as LDAC, QDAC, SVC, and ETC to perform multi-class texture classification and generate performance results.



Screenshot. 9.9: Dataset Train–Test Splitting Output

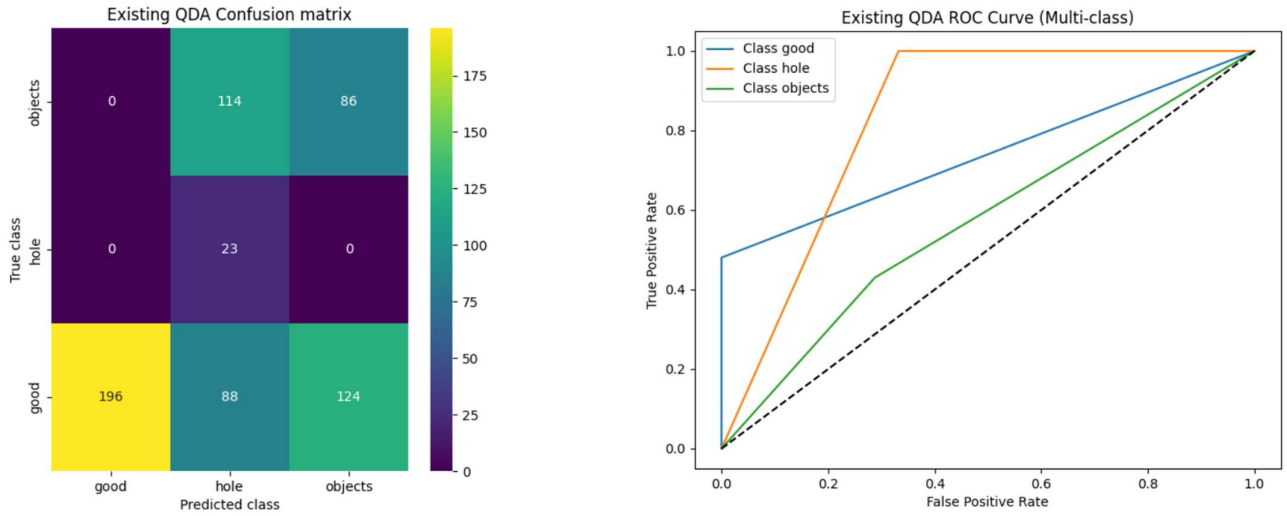


Screenshot. 9.10: Performance Evaluation of Existing LDAC Model (a) Confusion Matrix, (b) Multi-Class ROC Curve.

Screenshot. 9.10 (a) illustrates the confusion matrix obtained for the Existing LDAC during the texture classification experiment. The confusion matrix represents the relationship between the actual class labels and the predicted class labels produced by the classifier. It provides a detailed view of how correctly the classifier identifies each texture category and where misclassifications occur among the classes. The matrix highlights the distribution of predictions across categories such as good, hole, and objects, allowing the evaluation of classification accuracy for each class. This representation helps in understanding the strengths

and limitations of the LDAC model in distinguishing between different texture patterns. The confusion matrix is an important tool for analyzing classification performance and identifying potential improvements in the learning process.

Screenshot . 9.10 (b) depicts the multi-class ROC curve generated for the Existing LADC. The ROC curve illustrates the relationship between the true positive rate and the false positive rate for each texture class across different decision thresholds. Separate curves are plotted for each class to evaluate how effectively the classifier distinguishes between the categories. The closer the curves are to the upper-left region of the graph, the better the classification performance. This visualization helps measure the discriminative ability of the classifier and provides insights into the model's ability to correctly classify texture patterns. The ROC analysis complements the confusion matrix by offering a graphical interpretation of classification effectiveness.

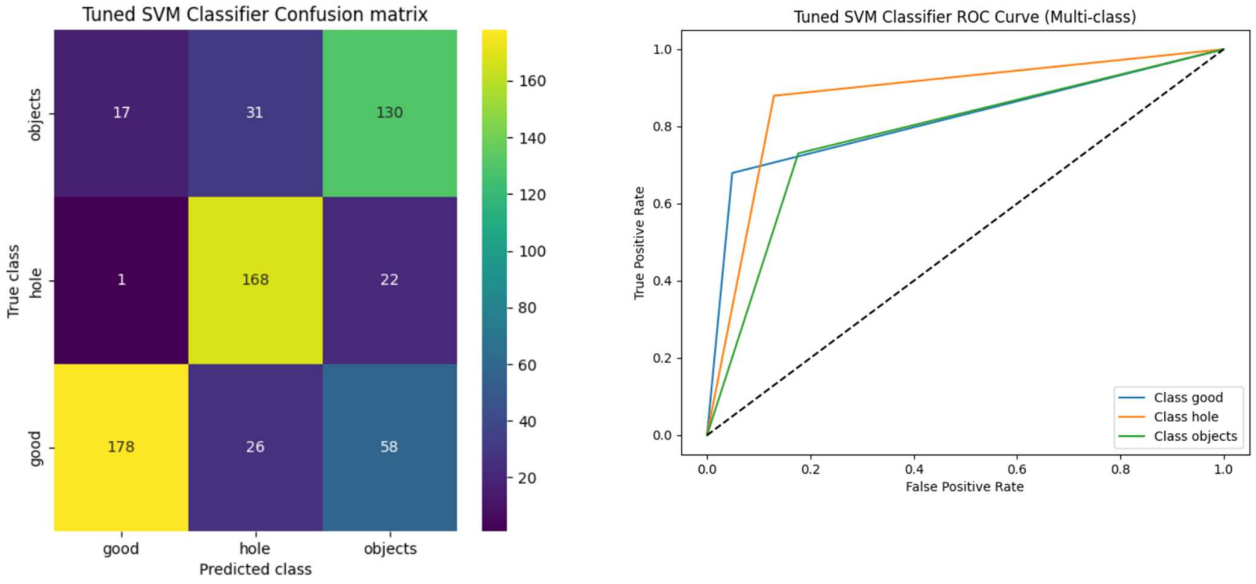


Screenshot. 9.11: Performance Evaluation of Existing QDAC Model (a) Confusion Matrix, (b) Multi-Class ROC Curve.

Screenshot. 9.11 (a) illustrates the confusion matrix obtained for the Existing QDAC during the texture classification experiment. The confusion matrix presents the comparison between the actual texture classes and the predicted classes generated by the classifier. It provides detailed information about the number of correctly classified samples as well as the misclassified instances among the categories such as good, hole, and objects. This

representation helps in evaluating how effectively the QDAC model distinguishes between different texture patterns present in the dataset. By analyzing the distribution of values within the matrix, it becomes possible to understand the classification strengths and areas where prediction errors occur. The confusion matrix serves as an important evaluation tool for assessing the overall performance of the classifier.

Screenshot. 9.11 (b) depicts the multi-class ROC curve generated for the Existing QDAC. The ROC curve represents the relationship between the true positive rate and the false positive rate for each texture category at different classification thresholds. Separate curves are plotted for each class to evaluate the classifier's ability to discriminate between different texture patterns. The graphical representation provides insight into how well the model distinguishes the classes across various decision boundaries. Curves that approach the upper-left corner indicate stronger classification capability. This ROC analysis complements the confusion matrix by providing a graphical assessment of the classifier's discriminative performance.

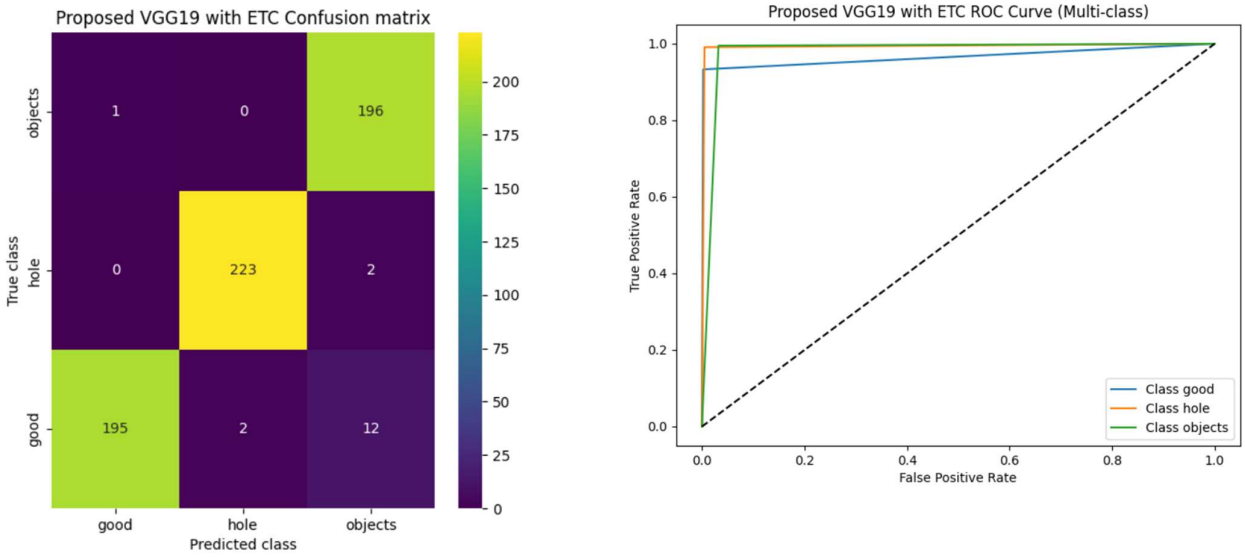


Screenshot. 9.12: Performance Evaluation of Existing SVM Model (a) Confusion Matrix, (b) Multi-Class ROC Curve.

Screenshot. 9.12 (a) illustrates the confusion matrix obtained for the Tuned SVC during the texture classification experiment. The confusion matrix provides a detailed representation of the relationship between the actual class labels and the predicted class labels produced by the classifier. It shows how effectively the classifier identifies the different texture categories such

as good, hole, and objects. The matrix highlights the number of correctly classified samples along with the instances where misclassification occurs between the classes. This representation helps evaluate the classification capability of the tuned SVC model in distinguishing complex texture patterns. The confusion matrix is therefore useful for analyzing the strengths and limitations of the classifier within the multi-class texture classification framework.

Screenshot. 9.12 (b) depicts the multi-class ROC curve generated for the Tuned SVC. The ROC curve presents the relationship between the true positive rate and the false positive rate for each texture class under different decision thresholds. Separate curves are plotted for each category to illustrate the classifier's ability to distinguish between the classes. The closer the curves approach the upper-left region of the graph, the stronger the classification performance of the model. This graphical representation provides insight into the discriminative capability of the SVC classifier for multi-class texture recognition. The ROC analysis complements the confusion matrix by offering a visual evaluation of the model's overall predictive performance.



Screenshot. 9.13: Performance Evaluation of Existing ETC Model (a) Confusion Matrix, (b) Multi-Class ROC Curve.

Screenshot. 9.13 (a) illustrates the confusion matrix obtained for the proposed classification approach that integrates VGG19 feature extraction with the ETC. The confusion matrix

represents the relationship between the actual texture categories and the predicted categories generated by the classifier. It provides detailed information about the correct classifications and the misclassifications among the texture classes such as good, hole, and objects. This representation highlights the ability of the proposed model to correctly distinguish between different surface texture patterns present in the dataset. The distribution of values in the matrix indicates improved classification accuracy compared to the baseline models. This evaluation helps demonstrate the effectiveness of combining deep feature extraction with an ensemble learning classifier for multi-class texture classification.

Screenshot. 9.13 (b) depicts the multi-class ROC curve generated for the proposed VGG19 with ETC model. The ROC curve illustrates the relationship between the true positive rate and the false positive rate for each texture class across different classification thresholds. Separate curves are plotted for each category to evaluate how effectively the proposed model discriminates between the classes. The curves approaching the upper-left region of the graph indicate strong classification capability and improved predictive performance. This graphical representation highlights the robustness of the proposed approach in identifying texture categories with higher accuracy. The ROC analysis complements the confusion matrix by providing a visual assessment of the classifier's discriminative performance across multiple classes.



Screenshot. 9.14: Test Image Prediction Interface (Laptop 2 Execution)

Screenshot. 9.14 illustrates the prediction interface used for classifying a test image within the texture classification framework. This stage represents the final inference phase where a new unseen image is provided to the trained classification model for analysis. The system processes the selected image through the same preprocessing and VGG19 feature extraction pipeline used

during the training phase. The extracted feature vector is then passed to the trained classifier, which generates the predicted texture category. The interface displays the prediction output for the uploaded test image, allowing users to observe the classification result produced by the model. This step demonstrates the practical deployment of the trained system for real-time texture recognition and verification of model performance on unseen data.



Screenshot. 9.15: Prediction Result – Classified as Hole

Screenshot. 9.15 illustrates the final prediction result obtained from the trained texture classification framework using a test image from the TILDA dataset. During this stage, the input image is processed through the same preprocessing and deep feature extraction pipeline using the VGG19 model. The extracted feature vector is then provided to the trained classifier, specifically the ETC, which determines the most appropriate texture category for the given image. The predicted output is displayed directly on the image to clearly indicate the classification result generated by the system. This visual representation confirms the ability of the trained model to analyze unseen images and correctly identify the texture class. The prediction stage demonstrates the practical applicability of the developed framework for automated texture recognition and surface defect identification.



Screenshot. 9.16: Prediction Result – Classified as Objects

Screenshot. 9.16 illustrates another prediction result generated by the trained texture classification framework using a test image from the TILDA dataset. In this stage, the input image is processed through the preprocessing pipeline followed by deep feature extraction using the VGG19 convolutional neural network. The extracted feature vector is then provided to the trained ETC, which analyses the visual patterns present in the image to determine the appropriate texture category. The predicted class label is displayed directly on the image to clearly indicate the classification outcome. This result demonstrates the capability of the trained model to identify different texture conditions present in unseen images. The prediction output confirms the effectiveness of the integrated feature extraction and machine learning classification approach for automated texture recognition.

9.4 Comparative Analysis

The comparative analysis evaluates the performance of different machine learning classifiers used in the texture classification framework. . Each model is trained using the deep feature vectors extracted through the VGG19 transfer learning model. The comparison is performed using standard evaluation metrics such as accuracy, precision, recall, F1-score, and ROC-AUC. The comparative study therefore provides insights into the performance improvements achieved by the proposed approach.

The comparative analysis results presented in the table 3 highlight the performance differences among the implemented machine learning classifiers. The LDAC achieved an accuracy of 90.97%, demonstrating strong performance in classifying the texture categories with balanced precision, recall, and F-score values. The QDAC produced a comparatively lower accuracy of 48.34%, indicating that this model struggled to effectively distinguish between the texture classes in the dataset. The SVC showed moderate performance with an accuracy of 75.44%, providing better classification capability than QDAC but still lower than LDAC. In contrast, the proposed ETC achieved the highest accuracy of 97.31%, outperforming all other models in terms of precision, recall, and F-score. The results indicate that the ensemble learning capability of ETC enables more accurate identification of complex texture patterns.

Model	Accuracy (%)	Precision (%)	Recall (%)	F-Score (%)
LDA C	90.97	91.10	91.19	90.87
QDAC	48.34	50.39	63.68	41.80
SVC	75.44	75.80	76.31	75.17
ETC	97.31	97.31	97.30	97.24

Table 3**:** Comparative Performance Analysis of Machine Learning Classifiers for Texture Classification.

CHAPTER 10

CONCLUSION AND FUTURE SCOPE

10.1 Conclusion

This research presents an intelligent texture classification framework for analysing industrial surface images using transfer learning and machine learning techniques. The system integrates VGG19-based deep feature extraction with multiple classifiers including LDAC, QDAC, SVC, and ETC. Experimental results demonstrate that the proposed approach significantly improves classification performance compared to traditional classifiers. Among the evaluated models, LDAC achieved an accuracy of 90.97%, SVC obtained 75.44%, and QDAC produced 48.34% accuracy. The proposed VGG19 with ETC model achieved the highest classification accuracy of 97.31%, along with improved precision, recall, and F-score values. The use of deep feature extraction combined with an ensemble classifier enables more effective identification of texture categories such as good, hole, and objects. The experimental results confirm that the proposed framework provides reliable and accurate texture classification suitable for automated industrial inspection applications.

10.2 Future Scope

The system can be further enhanced by incorporating advanced deep learning architectures such as ResNet or EfficientNet to improve feature representation. Future work may also explore real-time industrial inspection systems by integrating the framework with automated camera-based monitoring systems and edge computing platforms.

REFERENCES

[1] Hu, Q.; Wu, W.; Xia, T.; Yu, Q.; Yang, P.; Li, Z.; Song, Q. Exploring the use of google earth imagery and object-based methods in land use/cover mapping. Remote Sens. 2013, 5, 6026–6042.

[2] Toth, C.; Jóźków, G. Remote sensing platforms and sensors: A survey. ISPRS J. Photogramm. Remote Sens. 2016, 115, 22–36.

[3] Hoogendoorn, S.P.; Van Zuylen, H.J.; Schreuder, M.; Gorte, B.; Vosselman, G. Microscopic traffic data collection by remote sensing. Transp. Res. Rec. 2003, 1855, 121–128.

[4] Valavanis, K.P. Advances in Unmanned Aerial Vehicles: State of the Art and the Road to Autonomy; Springer Science & Business Media: Berlin, Germany, 2008; ISBN 978-1-4020-6114-1.

[5] Sheppard, C.; Rahnemoonfar, M. Real-time scene understanding for UAV imagery based on deep convolutional neural networks. In Proceedings of the 2017 IEEE International Geoscience and Remote Sensing Symposium (IGARSS), Fort Worth, TX, USA, 23–28 July 2017; pp. 2243–2246.

[6] Bazi, Y.; Bashmal, L.; Rahhal, M.M.A.; Dayil, R.A.; Ajlan, N.A. Vision Transformers for Remote Sensing Image Classification. Remote Sens. 2021, 13, 516. https://doi.org/10.3390/rs13030516.

[7] Carrilho, R.; Yaghoubi, E.; Lindo, J.; Hambarde, K.; Proença, H. Toward Automated Fabric Defect Detection: A Survey of Recent Computer Vision Approaches. Electronics 2024, 13, 3728. https://doi.org/10.3390/electronics13183728.

[8] Wang, R.; Ma, L.; He, G.; Johnson, B.A.; Yan, Z.; Chang, M.; Liang, Y. Transformers for Remote Sensing: A Systematic Review and Analysis. Sensors 2024, 24, 3495. https://doi.org/10.3390/s24113495.

[9] Wang, Y.; Deng, Y.; Zheng, Y.; Chattopadhyay, P.; Wang, L. Vision Transformers for Image Classification: A Comparative Survey. Technologies 2025, 13, 32. https://doi.org/10.3390/technologies13010032.

[10] Aleissaee, A.A.; Kumar, A.; Anwer, R.M.; Khan, S.; Cholakkal, H.; Xia, G.-S.; Khan, F.S. Transformers in Remote Sensing: A Survey. Remote Sens. 2023, 15, 1860. https://doi.org/10.3390/rs15071860.

[11] Tombe, R.; Viriri, S. Remote Sensing Image Scene Classification: Advances and Open Challenges. Geomatics 2023, 3, 137-155. https://doi.org/10.3390/geomatics3010007.

[12] Mao, M.; Hong, M. YOLO Object Detection for Real-Time Fabric Defect Inspection in the Textile Industry: A Review of YOLOv1 to YOLOv11. Sensors 2025, 25, 2270. https://doi.org/10.3390/s25072270.

[13] Zhang, J.; Zhao, H.; Li, J. TRS: Transformers for Remote Sensing Scene Classification. Remote Sens. 2021, 13, 4143. https://doi.org/10.3390/rs13204143.

[14] Li, Y.; Zhang, X.; Zhang, T.; Wang, G.; Wang, X.; Li, S. A Patch-Level Region-Aware Module with a Multi-Label Framework for Remote Sensing Image Captioning. Remote Sens. 2024, 16, 3987. https://doi.org/10.3390/rs16213987.

[15] Zhang, D.; Ma, W.; Jiao, L.; Liu, X.; Yang, Y.; Liu, F. Multiple Hierarchical Cross-Scale Transformer for Remote Sensing Scene Classification. Remote Sens. 2025, 17, 42. https://doi.org/10.3390/rs17010042.

[16] Lu, G.; He, X.; Wang, Q.; Shao, F.; Wang, H.; Wang, J. A Novel Multi-Scale Transformer for Object Detection in Aerial Scenes. Drones 2022, 6, 188. https://doi.org/10.3390/drones6080188.

[17] Qiao, Q.; Li, Y.; Lv, H. Open-Pit Mining Area Extraction Using Multispectral Remote Sensing Images: A Deep Learning Extraction Method Based on Transformer. Appl. Sci. 2024, 14, 6384. https://doi.org/10.3390/app14146384.

[18] Basit, A.; Siddique, M.A.; Bhatti, M.K.; Sarfraz, M.S. Comparison of CNNs and Vision Transformers-Based Hybrid Models Using Gradient Profile Loss for Classification of Oil Spills in SAR Images. Remote Sens. 2022, 14, 2085. https://doi.org/10.3390/rs14092085.

[19] Marjani, M.; Mahdianpari, M.; Mohammadimanesh, F.; Gill, E.W. CVTNet: A Fusion of Convolutional Neural Networks and Vision Transformer for Wetland Mapping Using Sentinel-1 and Sentinel-2 Satellite Data. Remote Sens. 2024, 16, 2427. https://doi.org/10.3390/rs16132427.

[20] Smith, A.D.; Du, S.; Kurien, A. Vision Transformers for Anomaly Detection and Localisation in Leather Surface Defect Classification Based on Low-Resolution Images and a Small Dataset. Appl. Sci. 2023, 13, 8716. https://doi.org/10.3390/app13158716.

[21] Zheng, F.; Lin, S.; Zhou, W.; Huang, H. A Lightweight Dual-Branch Swin Transformer for Remote Sensing Scene Classification. Remote Sens. 2023, 15, 2865. https://doi.org/10.3390/rs15112865.

[22] Tao, L.; Zhang, H.; Jing, H.; Liu, Y.; Yan, D.; Wei, G.; Xue, X. Advancements in Vision–Language Models for Remote Sensing: Datasets, Capabilities, and Enhancement Techniques. Remote Sens. 2025, 17, 162. https://doi.org/10.3390/rs17010162.

International Journal of
DATA SCIENCE AND IOT MANAGEMENT SYSTEM
Peer Reviewed, Referred & Indexed Journal
ISSN: 3068-272X www.ijdim.com Original Research Paper

Transfer Learning-based Ensemble Classifier for Multi-Class Texture Classification in Remote Sensing

G. Sujatha[1*], Syeda Sanobar Ali[2], Patimeeni Shiva Mukesh[2], Alval Abhilash Gou[2]
[1]Assistant Professor, [2]UG Student, [1,2]Department of Computer Science and Engineering (AI & ML),
[1,2]Kommuri Pratap Reddy Institute of Technology, Ghanpur, Ghatkesar, 501301, Telangana, India.
*Correspondence: G. Sujatha (gsujatha39@gmail.com)

Abstract
Automated texture classification has become an important area of research in computer vision and industrial inspection systems, where accurate surface analysis is required to maintain product quality and detect defects during manufacturing. In earlier industrial practices, surface inspection was performed manually by human experts who visually examined materials to identify defects such as holes, cracks, and foreign objects. Although manual inspection was widely used, it was often slow, inconsistent, and prone to human error. Later, traditional automated systems based on classical image processing techniques were introduced, but these methods relied heavily on handcrafted features and simple classification approaches, which limited their ability to capture complex texture patterns. As industrial datasets became larger and more diverse, these limitations created a need for intelligent systems capable of extracting meaningful visual features and performing reliable multi-class classification. To address these challenges, this study presents a transfer learning–based texture classification framework that combines deep feature extraction with machine learning techniques. In the proposed approach, a pretrained Visual Geometry Group 19 (VGG19) is used to extract high-level visual features from industrial texture images. The extracted feature vectors are then used to train multiple machine learning classifiers including Linear Discriminant Analysis Classifier (LDAC), Quadratic Discriminant Analysis Classifier (QDAC), Support Vector Classifier (SVC), and Extra Trees Classifier (ETC). Among these models, the proposed VGG19 with ETC classifier achieved the highest classification accuracy of 97.31%, demonstrating its strong capability in distinguishing between different texture categories such as good, hole, and objects.

Keywords: Texture Classification, Industrial Inspection, Computer Vision, Transfer Learning, Deep Feature Extraction, Visual Geometry Group 19 (VGG19).

1. Introduction

Remote sensing (RS) is the science of collecting information about objects without any direct physical contact, typically through a satellite, aircraft or unmanned aerial vehicle (UAV). Examples of applications of remote sensing include geological survey, environment testing, oil exploration, traffic management, earthquake prediction, and water conservancy construction. Remote-sensing images have improved in both spatial and temporal resolutions with the evolution of satellite sensors, which provides opportunities in resolving fine details on the earth's surface as shown in Figure 1. Satellites such as MODIS (1 km × 1 km) offering thermal data with high temporal resolution suffer from low spatial resolution. Landsat, on the other hand, offers small-scale variations of 100–200 m but with very low temporal resolution. The new generation of satellites can deliver very high spectral and spatial images; for example, IKONOS-2 generates images with 4-band multispectral resolution and spatial resolution from 2.5 to 4 m. Unmanned aerial vehicles (UAVs) present an improved solution of remote-sensing acquisition platforms, which witnessed a high level of growth in past years and are used widely for fire detection, surveillance mapping, and landslide monitoring, among other uses [2]. UAVs have several advantages over satellite and aerial images. First, they are easier to deploy to satisfy the requirements of rapid monitoring, assessment, and mapping. They can work at lower altitudes compared to the piloted aircraft, which provides spatial resolution at the centimeters level.

International Journal of
DATA SCIENCE AND IOT MANAGEMENT SYSTEM
Peer Reviewed, Referred & Indexed Journal
ISSN: 3068-272X www.ijdim.com Original Research Paper

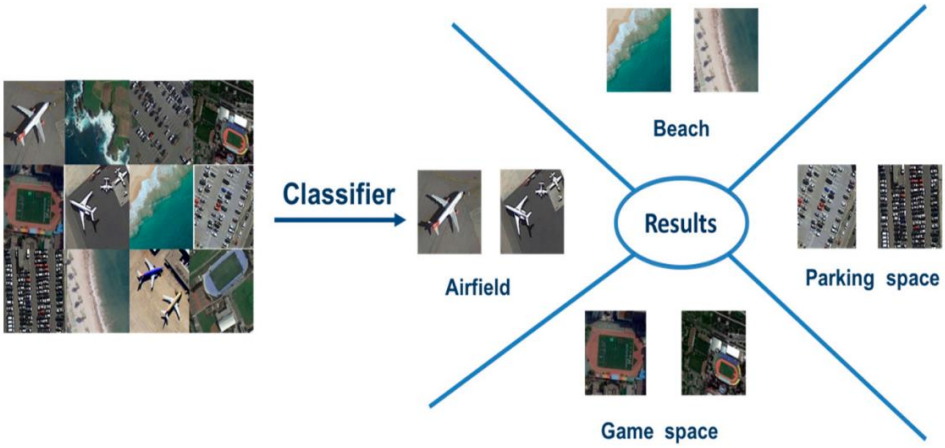


Figure. 1: Remote sensing scene classification.

They can fly any time the weather permits, leading to improvements in temporal resolution. As the spatial resolution increases, images are likely to contain noisy and outlying descriptors. A recent study [3] proposed a convolutional neural network (CNN) to classify images captured by a camera mounted on a UAV. CNN model to classify a digital surface model beside UAV images [4]. combined CNNs with object-based image analysis (OBIA) for land cover classification using Multiview data. The two-branch neural network to assign multiple class labels to UAV imagery [5].

2. Literature Survey

Bazi, et al. [6] proposed a remote-sensing scene-classification method based on vision transformers. These types of networks, which are now recognized as state-of-the-art models in natural language processing, do not rely on convolution layers as in standard convolutional neural networks (CNNs). Instead, they use multihead attention mechanisms as the main building block to derive long-range contextual relation between pixels in images. In a first step, the images under analysis are divided into patches, then converted to sequence by flattening and embedding. Carrilho, et al. [7] reviewed, they summarized the most important advancements in the last 5 years and focus mostly on machine learning-based approaches. They also outline the most promising avenues of research in the future.

Wang, et al. [8] developed and found that transformers need more parameters than CNNs. Additionally, further research is also needed regarding inference speed to improve transformers' performance. It was determined that the most common application scenes for transformers in our database are urban, farmland, and water bodies. They also found that transformers are employed in the natural sciences such as agriculture and environmental protection rather than the humanities or economics. Finally, this work summarizes the analysis results of transformers in remote sensing obtained during the research process and provides a perspective on future directions of development. Wang, et al. [9] introduced the fundamental concepts of transformers and highlight the first successful Vision Transformer (ViT). Building on the ViT, they reviewed subsequent improvements and optimizations introduced for image classification tasks. They then compare the strengths and limitations of these transformer-based models against classic CNNs through experiments. Finally, they explored key challenges and potential future directions for image classification transformers.

Aleissaee, et al. [10] reviewed the remote sensing community and has also witnessed an increased exploration of vision transformers for a diverse set of tasks. Although several surveys have focused on transformers in computer vision in general, to the best of their knowledge they are the first to present a systematic review of recent advances based on transformers in remote sensing. Their survey covers more than 60 recent transformer-based methods for different remote sensing problems in sub-areas of

International Journal of
DATA SCIENCE AND IOT MANAGEMENT SYSTEM
Peer Reviewed, Referred & Indexed Journal
ISSN: 3068-272X www.ijdim.com Original Research Paper

remote sensing: very high-resolution (VHR), hyperspectral (HSI) and synthetic aperture radar (SAR) imagery. They concluded the survey by discussing different challenges and open issues of transformers in remote sensing. Tombe, et al. [11] presented a comprehensive review of the developments of various computer vision methods in remote sensing. There is currently an increase of remote sensing datasets with diverse scene semantics; this renders computer vision methods challenging to characterize the scene images for accurate scene classification effectively. This paper presented technology breakthroughs in deep learning and discusses their artificial intelligence open-source software implementation framework capabilities. Further, this paper discusses the open gaps/opportunities that need to be addressed by remote sensing communities.

Mao, et al. [12] presented this work comprehensively comparing the entire YOLO family, highlighting key innovations and their practical implications. They also discuss the challenges, including dataset limitations, domain generalization, and computational constraints, proposing future solutions such as synthetic data generation, federated learning, and edge AI deployment. By bridging the gap between academic advancements and industrial applications, this review is a practical guide for selecting and optimizing YOLO models for fabric inspection, paving the way for intelligent quality control systems. Zhang, et al. [13] proposed a new remote sensing scene classification method, Remote Sensing Transformer (TRS), a powerful "pure CNNs → Convolution + Transformer → pure Transformers" structure. First, they integrate self-attention into ResNet in a novel way, using our proposed Multi-Head Self-Attention layer instead of 3 × 3 spatial revolutions in the bottleneck. Then they connect multiple pure Transformer encoders to further improve the representation learning performance completely depending on attention. Finally, use a linear classifier for classification. They train our model on four public remote sensing scene datasets: UC-Merced, AID, NWPU-RESISC45, and OPTIMAL-31. The experimental results show that TRS exceeds the state-of-the-art methods and achieves higher accuracy.

Li, et al. [14] investigated unexplored ideas for remote sensing image captioning task, using a novel patch-level region-aware module with a multi-label framework. Due to an overhead perspective and a significantly larger scale in RSIs, a patch-level region-aware module is designed to filter the redundant information in the RSI scene, which benefits the Transformer-based decoder by attaining improved image perception. Technically, the trainable multi-label classifier capitalizes on semantic features as supplementary to the region-aware features. Moreover, modeling the inner relations of inputs is essential for understanding the RSI. Zhang, et al. [15] proposed a multiple hierarchical cross-scale Transformer model that efficiently combines the Transformer model with CNNs and is specifically designed for complex remote sensing scene classification. Firstly, a feature pyramid network with attention aggregation extracts the multi-scale base features. Then, these base features are fed into the proposed multi-scale channel Transformer (MSCT) module to derive the global features with channel-wise attention. Additionally, the base features are also fed into the proposed hierarchical cross-scale Transformer (HCST) module, which can obtain multi-level cross-scale representations.

Lu, et al. [16] proposed a novel aerial object detection framework called DFCformer. DFCformer is mainly composed of three parts: the backbone network DMViT, which introduces deformation patch embedding and multi-scale adaptive self-attention to capture sufficient features of the objects; FRGC guides feature interaction layer by layer to break the barriers between feature layers and improve the information discrimination and processing ability of multi-scale critical features; CAIM adopts an attention mechanism to fuse multi-scale features to perform hierarchical reasoning on the relationship

International Journal of
DATA SCIENCE AND IOT MANAGEMENT SYSTEM
Peer Reviewed, Referred & Indexed Journal
ISSN: 3068-272X www.ijdim.com Original Research Paper

between different levels and fully utilize the complementary information in multi-scale features. Extensive experiments have been conducted on the FAIR1M dataset, and DFC former shows its advantages by achieving the highest scores with stronger scene adaptability.

3. Proposed Methodology

The proposed methodology follows a structured framework for developing an intelligent image analysis system capable of performing multi-class texture classification in industrial environments shown in figure 2. The methodology begins with dataset acquisition and image preprocessing, followed by deep feature extraction using transfer learning techniques. These extracted features are then utilized by multiple machine learning classifiers to analyses surface textures and identify different defect categories. The system integrates classical classification models with deep feature extraction to improve classification accuracy and reliability. A graphical interface allows users to interact with the system, perform dataset uploads, execute model training, and view classification results. This structured workflow ensures efficient data handling, model evaluation, and prediction generation for automated texture analysis.

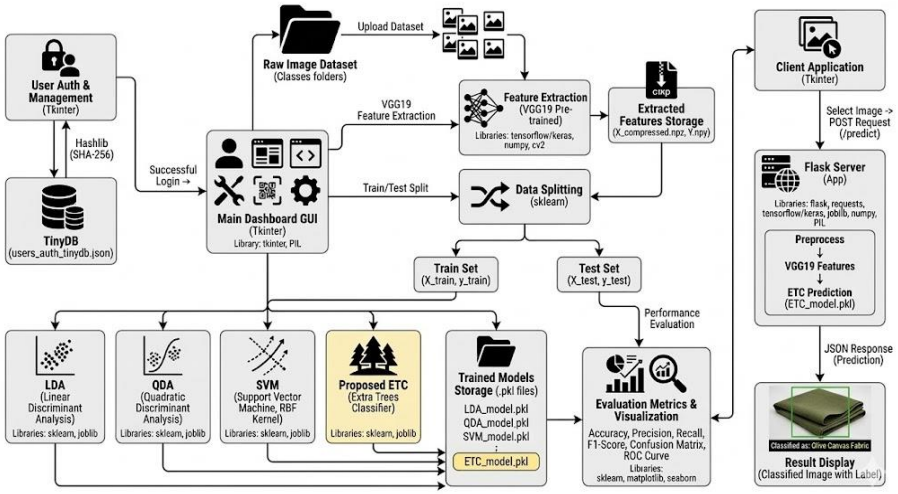


Figure. 2: System architecture of patch-level texture classification.

User Interface (Tkinter GUI)

- The user interacts with the system through a custom graphical interface developed using Tkinter.
- The interface facilitates critical workflow stages, including dataset uploading, feature extraction, model training, and performance evaluation.
- Users can visualize real-time classification results, confusion matrices, and detailed performance metrics.
- All GUI-triggered actions are communicated to backend modules that orchestrate the data analysis pipeline.

Dataset Input (Image Dataset)

- The system utilizes a structured dataset of industrial texture images categorized into specific class folders.
- These categories represent surface conditions such as normal surfaces, holes, or foreign objects.
- Images are loaded into the system and prepared for the preprocessing pipeline to ensure consistency across the dataset.

Image Preprocessing

International Journal of
DATA SCIENCE AND IOT MANAGEMENT SYSTEM
Peer Reviewed, Referred & Indexed Journal
ISSN: 3068-272X www.ijdim.com Original Research Paper

- To ensure uniform dimensions for the neural network, all uploaded images are resized to a standard resolution.
- Images are converted into multidimensional arrays and normalized to optimize the efficiency of the feature extraction process.
- This standardized data format is then forwarded to the deep learning feature extractor.

Feature Extraction using VGG19

- A pretrained VGG19 convolutional neural network serves as the backbone for capturing high-level visual patterns.
- The final fully connected layers are truncated, allowing the convolutional base to act as a pure feature extractor.
- **Global Average Pooling** is applied to the output to generate compact numerical feature vectors that represent structural details and spatial patterns.

Train–Test Data Splitting

- The extracted feature vectors are partitioned into distinct training and testing subsets.
- The training set allows the classifiers to learn specific texture patterns, while the testing set provides an unbiased evaluation of the models' predictive power.
- This separation is crucial for validating the generalizability of the system on unseen industrial data.

Existing Baseline Models (LDAC, QDAC, SVC)

- The feature vectors are processed by several classical statistical and machine learning classifiers to establish a performance benchmark:
  - **LDAC:** Focuses on maximizing the linear separability between different surface classes.
  - **QDAC:** Models are more complex, quadratic decision boundaries for higher-dimensional data.
  - **SVC:** Utilizes kernel functions to identify the optimal hyperplane that separates texture categories.

Proposed Model (ETC)

- ETC is implemented as the core ensemble learning engine for the framework.
- It operates by constructing many randomized decision trees and aggregating their results to reach a final classification.
- By using randomized feature selection during tree construction, the model effectively reduces overfitting and improves the overall accuracy of texture classification.

Performance Evaluation and Result Analysis

- The outputs of both the baseline and proposed models are rigorously analyzed using a comprehensive suite of metrics.
- Evaluation includes Accuracy, Precision, Recall, and F1-score, alongside visual tools like Confusion Matrices and ROC Curves.
- This analysis identifies the most reliable classifier for specific industrial environments and surface types.

Prediction and Output Visualization

- In the final stage, the trained classifier is deployed to identify the class of new, unseen industrial images.
- New inputs pass through the same preprocessing and VGG19 feature extraction pipeline.
- The predicted class label is overlaid on the image and displayed within the Tkinter UI, allowing for rapid identification of surface conditions and defects.

International Journal of
DATA SCIENCE AND IOT MANAGEMENT SYSTEM
Peer Reviewed, Referred & Indexed Journal
ISSN: 3068-272X www.ijdim.com Original Research Paper

3.1 VGG19

The feature extraction stage plays a crucial role in transforming raw image data into meaningful representations that can be used for classification. In this study, a pretrained deep convolutional neural network is used to extract high-level visual features from industrial texture images. The network processes the input images through multiple convolutional and pooling layers to capture important spatial patterns such as edges, shapes, and texture structures. Instead of using the network for direct classification, the final classification layers are removed so that the deep network functions as a powerful feature extractor, as shown in fig 3. The resulting feature vectors represent the visual characteristics of each image and are later used by machine learning classifiers for texture classification.

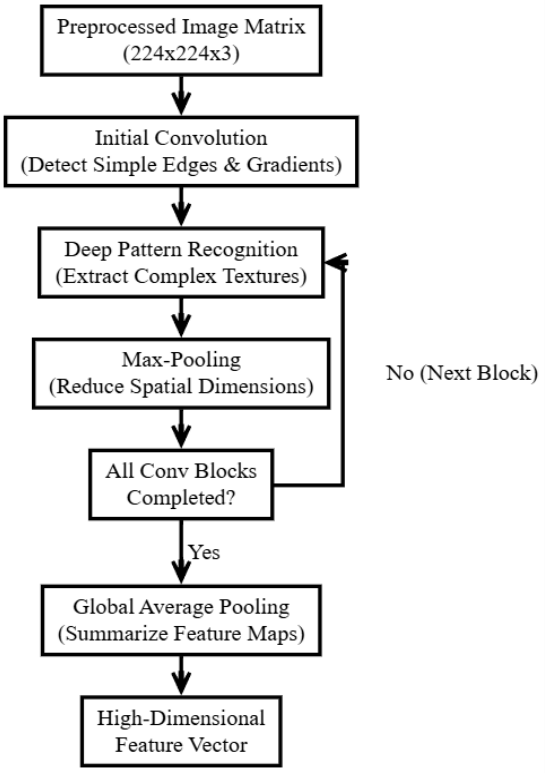


Figure. 3: Internal workflow of VGG19.

Input Image Preparation: The preprocessed images are provided as input to the deep convolutional network. Each image is resized to the required dimensions and formed into a numerical array structure. This standardized input ensures compatibility with the feature extraction model.

Convolutional Feature Learning: The input image passes through multiple convolutional layers within the network. These layers apply learnable filters that capture low-level visual patterns such as edges and color variations. As the layers progress, the network begins to identify more complex structures and texture patterns.

Activation and Non-Linear Transformation: After convolution operations, activation functions introduce non-linear transformations to the extracted features. This process enables the network to learn complex visual relationships within the image. The activation layers enhance the network's ability to represent intricate texture variations.

Spatial Down sampling through Pooling: Pooling layers are used to reduce the spatial dimensions of the feature maps generated by convolutional layers. This operation helps retain important visual information while reducing computational complexity. Pooling also improves the robustness of the feature representation against small image variations.

International Journal of
DATA SCIENCE AND IOT MANAGEMENT SYSTEM
Peer Reviewed, Referred & Indexed Journal
ISSN: 3068-272X www.ijdim.com Original Research Paper

Feature Vector Generation: After the convolution and pooling operations are completed, global average pooling is applied to convert the feature maps into a compact numerical feature vector. This feature vector summarizes the most important visual information extracted from the image. The generated features are then forwarded to machine learning classifiers for the classification stage.

4. Result Description

The results of the study demonstrate the effectiveness of the proposed texture classification framework in analyzing industrial surface images. The system processes the input dataset through preprocessing, deep feature extraction, and machine learning classification stages to generate accurate predictions. Experimental results are obtained by training multiple classifiers and evaluating their performance using standard evaluation metrics. The results provide insights into how effectively the classifiers distinguish between different texture categories such as normal surfaces, holes, and foreign objects. Performance comparisons among the implemented models highlight the strengths and limitations of each classifier. These outcomes help determine the most reliable model for accurate industrial texture classification.

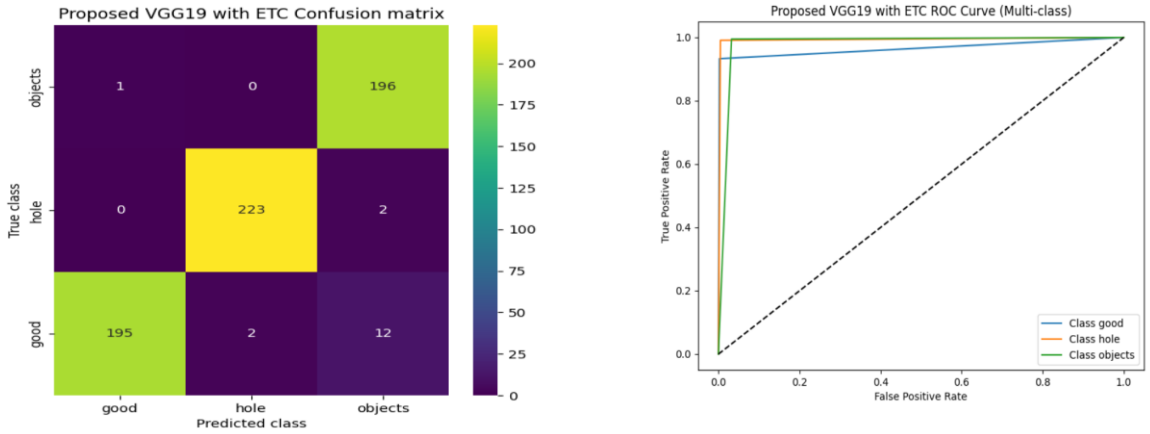


Figure 4: Performance Evaluation of Existing ETC Model (a) Confusion Matrix, (b) Multi-Class ROC Curve.

Figure 4(a) illustrates the confusion matrix obtained for the proposed classification approach that integrates VGG19 feature extraction with the ETC. The confusion matrix represents the relationship between the actual texture categories and the predicted categories generated by the classifier. It provides detailed information about the correct classifications and the misclassifications among the texture classes such as good, hole, and objects. This representation highlights the ability of the proposed model to correctly distinguish between different surface texture patterns present in the dataset. The distribution of values in the matrix indicates improved classification accuracy compared to the baseline models. This evaluation helps demonstrate the effectiveness of combining deep feature extraction with an ensemble learning classifier for multi-class texture classification.

Figure 4(b) depicts the multi-class ROC curve generated for the proposed VGG19 with ETC model. The ROC curve illustrates the relationship between the true positive rate and the false positive rate for each texture class across different classification thresholds. Separate curves are plotted for each category to evaluate how effectively the proposed model discriminates between the classes. The curves approaching the upper-left region of the graph indicate strong classification capability and improved predictive performance. This graphical representation highlights the robustness of the proposed approach in identifying texture categories with higher accuracy. The ROC analysis complements the confusion matrix by providing a visual assessment of the classifier's discriminative performance across multiple classes.

International Journal of
DATA SCIENCE AND IOT MANAGEMENT SYSTEM
Peer Reviewed, Referred & Indexed Journal
ISSN: 3068-272X www.ijdim.com Original Research Paper

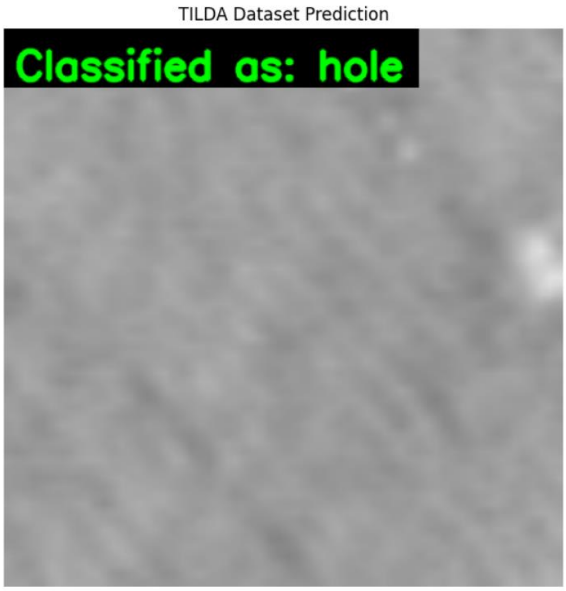


Figure. 5: Prediction result classified as Hole

Figure 5 illustrates the final prediction result obtained from the trained texture classification framework using a test image from the TILDA dataset. During this stage, the input image is processed through the same preprocessing and deep feature extraction pipeline using the VGG19 model. The extracted feature vector is then provided to the trained classifier, specifically the ETC, which determines the most appropriate texture category for the given image. The predicted output is displayed directly on the image to clearly indicate the classification result generated by the system. This visual representation confirms the ability of the trained model to analyze unseen images and correctly identify the texture class. The prediction stage demonstrates the practical applicability of the developed framework for automated texture recognition and surface defect identification.

Figure 6 illustrates another prediction result generated by the trained texture classification framework using a test image from the TILDA dataset. In this stage, the input image is processed through the preprocessing pipeline followed by deep feature extraction using the VGG19 convolutional neural network. The extracted feature vector is then provided to the trained ETC, which analyses the visual patterns present in the image to determine the appropriate texture category. The predicted class label is displayed directly on the image to clearly indicate the classification outcome. This result demonstrates the capability of the trained model to identify different texture conditions present in unseen images. The prediction output confirms the effectiveness of the integrated feature extraction and machine learning classification approach for automated texture recognition.

International Journal of
DATA SCIENCE AND IOT MANAGEMENT SYSTEM
Peer Reviewed, Referred & Indexed Journal
ISSN: 3068-272X www.ijdim.com Original Research Paper

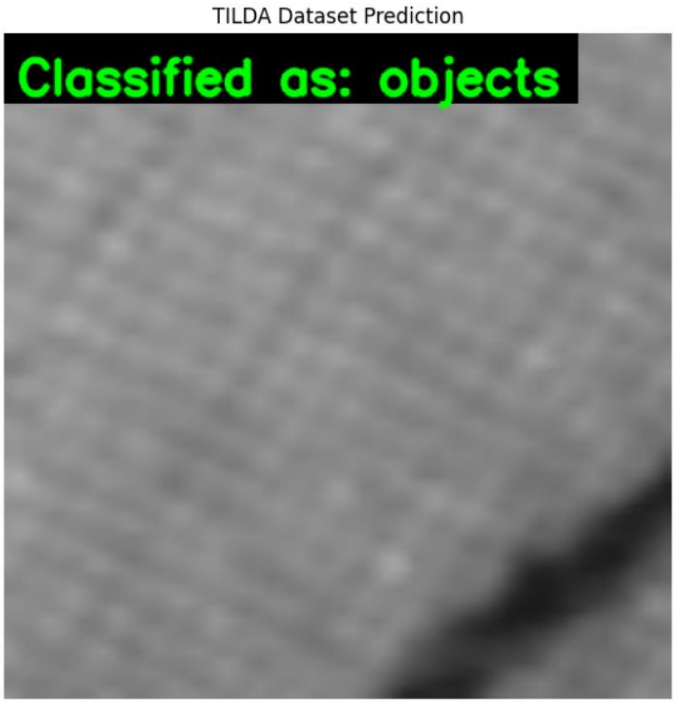


Figure. 6: Prediction result classified as Objects

The comparative analysis results presented in the table 1 highlight the performance differences among the implemented machine learning classifiers. The LDAC achieved an accuracy of 90.97%, demonstrating strong performance in classifying the texture categories with balanced precision, recall, and F-score values. The QDAC produced a comparatively lower accuracy of 48.34%, indicating that this model struggled to effectively distinguish between the texture classes in the dataset. The SVC showed moderate performance with an accuracy of 75.44%, providing better classification capability than QDAC but still lower than LDAC. In contrast, the proposed ETC achieved the highest accuracy of 97.31%, outperforming all other models in terms of precision, recall, and F-score. The results indicate that the ensemble learning capability of ETC enables more accurate identification of complex texture patterns.

Table 1**:** Comparative Performance Analysis of Machine Learning Classifiers for Texture Classification.

Model	Accuracy (%)	Precision (%)	Recall (%)	F-Score (%)
LDAC	90.97	91.10	91.19	90.87
QDAC	48.34	50.39	63.68	41.80
SVC	75.44	75.80	76.31	75.17
ETC	97.31	97.31	97.30	97.24

5. Conclusion

This research presents an intelligent texture classification framework for analyzing industrial surface images using transfer learning and machine learning techniques. The system integrates VGG19-based deep feature extraction with multiple classifiers including LDAC, QDAC, SVC, ETC. Experimental results demonstrate that the proposed approach significantly improves classification performance compared to traditional classifiers. Among the evaluated models, LDAC achieved an accuracy of 90.97%, SVC obtained 75.44%, and QDAC produced 48.34% accuracy. The proposed VGG19 with ETC model achieved the highest classification accuracy of 97.31%, along with improved precision,

recall, and F-score values. The use of deep feature extraction combined with an ensemble classifier enables more effective identification of texture categories such as good, hole, and objects. The experimental results confirm that the proposed framework provides reliable and accurate texture classification suitable for automated industrial inspection applications.

References

[1] Hu, Q.; Wu, W.; Xia, T.; Yu, Q.; Yang, P.; Li, Z.; Song, Q. Exploring the use of google earth imagery and object-based methods in land use/cover mapping. Remote Sens. 2013, 5, 6026–6042.

[2] Toth, C.; Jóźków, G. Remote sensing platforms and sensors: A survey. ISPRS J. Photogramm. Remote Sens. 2016, 115, 22–36.

[3] Hoogendoorn, S.P.; Van Zuylen, H.J.; Schreuder, M.; Gorte, B.; Vosselman, G. Microscopic traffic data collection by remote sensing. Transp. Res. Rec. 2003, 1855, 121–128.

[4] Valavanis, K.P. Advances in Unmanned Aerial Vehicles: State of the Art and the Road to Autonomy; Springer Science & Business Media: Berlin, Germany, 2008; ISBN 978-1-4020-6114-1.

[5] Sheppard, C.; Rahnemoonfar, M. Real-time scene understanding for UAV imagery based on deep convolutional neural networks. In Proceedings of the 2017 IEEE International Geoscience and Remote Sensing Symposium (IGARSS), Fort Worth, TX, USA, 23–28 July 2017; pp. 2243–2246.

[6] Bazi, Y.; Bashmal, L.; Rahhal, M.M.A.; Dayil, R.A.; Ajlan, N.A. Vision Transformers for Remote Sensing Image Classification. Remote Sens. 2021, 13, 516. https://doi.org/10.3390/rs13030516.

[7] Carrilho, R.; Yaghoubi, E.; Lindo, J.; Hambarde, K.; Proença, H. Toward Automated Fabric Defect Detection: A Survey of Recent Computer Vision Approaches. Electronics 2024, 13, 3728. https://doi.org/10.3390/electronics13183728.

[8] Wang, R.; Ma, L.; He, G.; Johnson, B.A.; Yan, Z.; Chang, M.; Liang, Y. Transformers for Remote Sensing: A Systematic Review and Analysis. Sensors 2024, 24, 3495. https://doi.org/10.3390/s24113495.

[9] Wang, Y.; Deng, Y.; Zheng, Y.; Chattopadhyay, P.; Wang, L. Vision Transformers for Image Classification: A Comparative Survey. Technologies 2025, 13, 32. https://doi.org/10.3390/technologies13010032.

[10] Aleissaee, A.A.; Kumar, A.; Anwer, R.M.; Khan, S.; Cholakkal, H.; Xia, G.-S.; Khan, F.S. Transformers in Remote Sensing: A Survey. Remote Sens. 2023, 15, 1860. https://doi.org/10.3390/rs15071860.

[11] Tombe, R.; Viriri, S. Remote Sensing Image Scene Classification: Advances and Open Challenges. Geomatics 2023, 3, 137-155. https://doi.org/10.3390/geomatics3010007.

[12] Mao, M.; Hong, M. YOLO Object Detection for Real-Time Fabric Defect Inspection in the Textile Industry: A Review of YOLOv1 to YOLOv11. Sensors 2025, 25, 2270. https://doi.org/10.3390/s25072270.

[13] Zhang, J.; Zhao, H.; Li, J. TRS: Transformers for Remote Sensing Scene Classification. Remote Sens. 2021, 13, 4143. https://doi.org/10.3390/rs13204143.

[14] Li, Y.; Zhang, X.; Zhang, T.; Wang, G.; Wang, X.; Li, S. A Patch-Level Region-Aware Module with a Multi-Label Framework for Remote Sensing Image Captioning. Remote Sens. 2024, 16, 3987. https://doi.org/10.3390/rs16213987.

[15] Zhang, D.; Ma, W.; Jiao, L.; Liu, X.; Yang, Y.; Liu, F. Multiple Hierarchical Cross-Scale Transformer for Remote Sensing Scene Classification. Remote Sens. 2025, 17, 42. https://doi.org/10.3390/rs17010042.

[16] Lu, G.; He, X.; Wang, Q.; Shao, F.; Wang, H.; Wang, J. A Novel Multi-Scale Transformer for Object Detection in Aerial Scenes. Drones 2022, 6, 188. https://doi.org/10.3390/drones6080188.